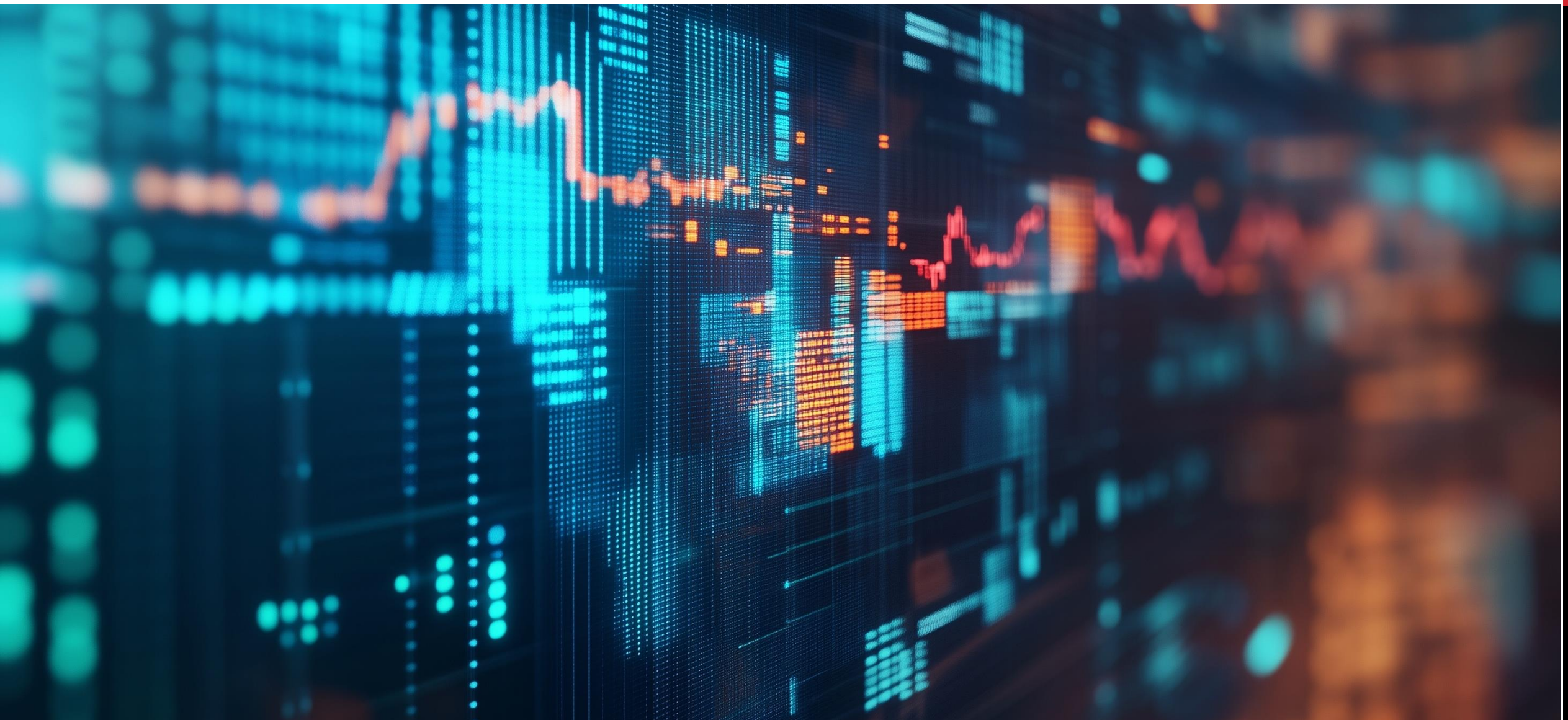
Day 1— Data Platforms, Integration & Migration



AGENDA

Day	Focus Area	Key Outcomes
DAY 1	**Data Platforms, Integration & Migration** • MongoDB Overview • Informatica ETL Fundamentals • Data Migration Use Cases • Hybrid Data Integration	Build strong foundation in data, ETL, migration, and hybrid integration
DAY 2	**Production Support Fundamentals** • On-call operations & SLA compliance • Incident management & triage • Documentation & reporting	Develop operational excellence and effective support practices
DAY 3	**Infrastructure Reliability & Automation** • Infrastructure reliability & failover testing • Automation of operational tasks • Monitoring & efficiency measurement	Ensure reliability, automate processes, and improve operational efficiency

COURSE INTRODUCTION

This program is designed to equip you with practical skills across modern data platforms, data integration, production support, and infrastructure reliability. Through concepts, real-world use cases, and hands-on labs, you will gain end-to-end expertise to drive enterprise data and operations excellence.

WHAT YOU WILL LEARN

- MongoDB fundamentals, data modeling, querying, aggregation, and performance optimization
- Informatica ETL concepts, mappings, transformations, workflows, and performance tuning
- Data migration methodologies, real-world use cases, and validation strategies
- Hybrid data integration patterns, architectures, and modern integration technologies
- Production support practices, SLAs, incident management, and operational excellence

MODULE 1 — MONGODB OVERVIEW

ENTERPRISE DATA LANDSCAPE

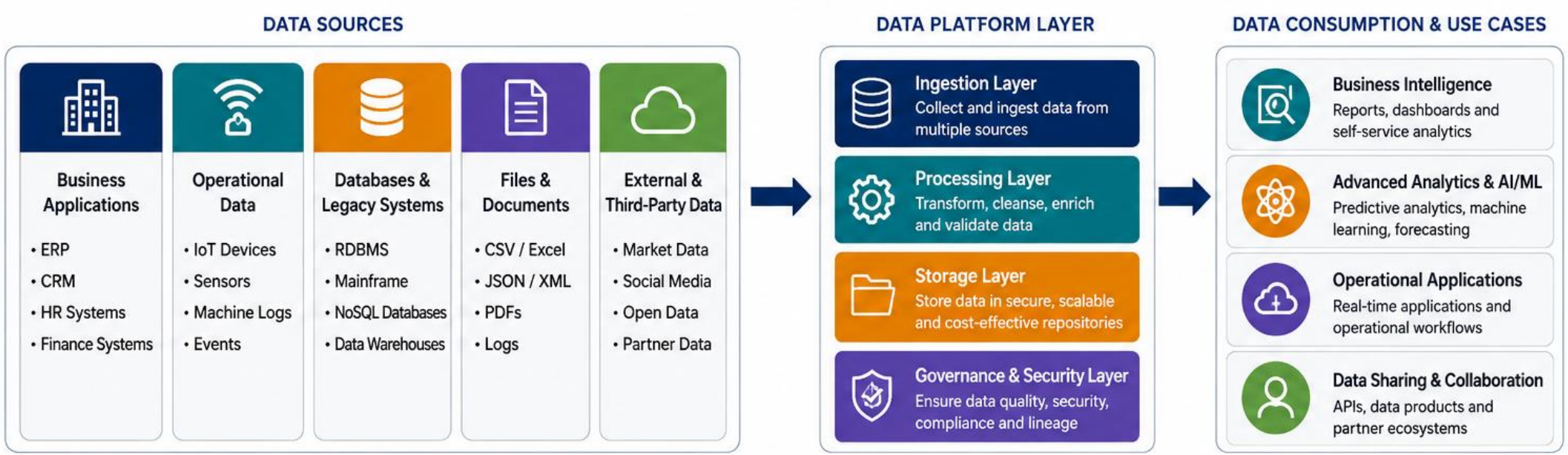
The foundation of modern data-driven organizations

Enterprises generate and consume data from a wide variety of sources. A well-designed data landscape enables unified access, trusted insights, and better business outcomes.

Why It Matters

- Drive Better Decisions
- Increase Efficiency
- Improve Data Quality
- Enable Scalability
- Support Innovation

ENTERPRISE DATA LANDSCAPE



RELATIONAL DATABASES VS NOSQL DATABASES

Relational databases excel in structures data and transactions, while NoSQL databases excel in scalability, flexibility, and handling large volumes of diverse data.

Why It Matters

- Optimize Performance
- Enable Scalability
- Reduce Costs
- Increase Flexibility
- Support Modern Applications

Relational Databases	Aspect	NoSQL Databases
• Structured data organized in tables • Fixed schema with predefined structure	**Data Model**	• Flexible data models (document, key-value, column, graph) • Schema-less or dynamic schema
• Schema is predefined • Altering schema can be complex	**Schema**	• Schema is flexible • Easy to evolve and adapt
• Scale vertically (using powerful servers) • Limited horizontal scaling	**Scalability**	• Scale horizontally (add more nodes) • Designed for large-scale distributed systems
• Uses SQL (Structured Query Language)	**Query Language**	• Uses various query languages or APIs (e.g., MongoDB Query, CQL, Cypher)
• ACID properties fully supported • Strong consistency	**Transactions**	• BASE model (Basically Available, Soft state, Eventually consistent) • Tunable consistency
• Financial systems, ERP, CRM • Applications with complex transactions	**Use Cases**	• Big data, real-time analytics, IoT, content management • Applications needing high scalability and flexibility
• Oracle, MySQL, SQL Server • PostgreSQL, DB2	**Examples**	• MongoDB, Cassandra, Redis, Couchbase • DynamoDB, Neo4j, HBase

WHY ORGANIZATIONS ADOPT MONGODB

Built for modern applications and data needs

MongoDB is a leading NoSQL database that helps organizations build scalable, high-performance applications faster. It is flexible, developer-friendly, and enterprise-ready.

Why It Matters

- Accelerate Application Delivery
- Improve Scalability and Performance
- Handle Diverse and Evolving Data
- Enhance Reliability and Availability
- Reduce Total Cost of Ownership

KEY REASONS TO ADOPT MONGODB

FLEXIBLE DATA MODEL

- Schema-less design
- Easily adapt to changing requirements
- Store complex, hierarchical data naturally

HIGH SCALABILITY

- Horizontal scaling with sharding
- Handle large volumes of data and traffic
- Built for growth

POWERFUL PERFORMANCE

- High throughput and low latency
- Rich indexing capabilities
- Optimized for modern workloads

DEVELOPER FRIENDLY

- Easy to learn and use
- Native support for popular languages
- Faster development cycles

ENTERPRISE READY

- High availability with replica sets
- Strong security and compliance
- Global ecosystem and support

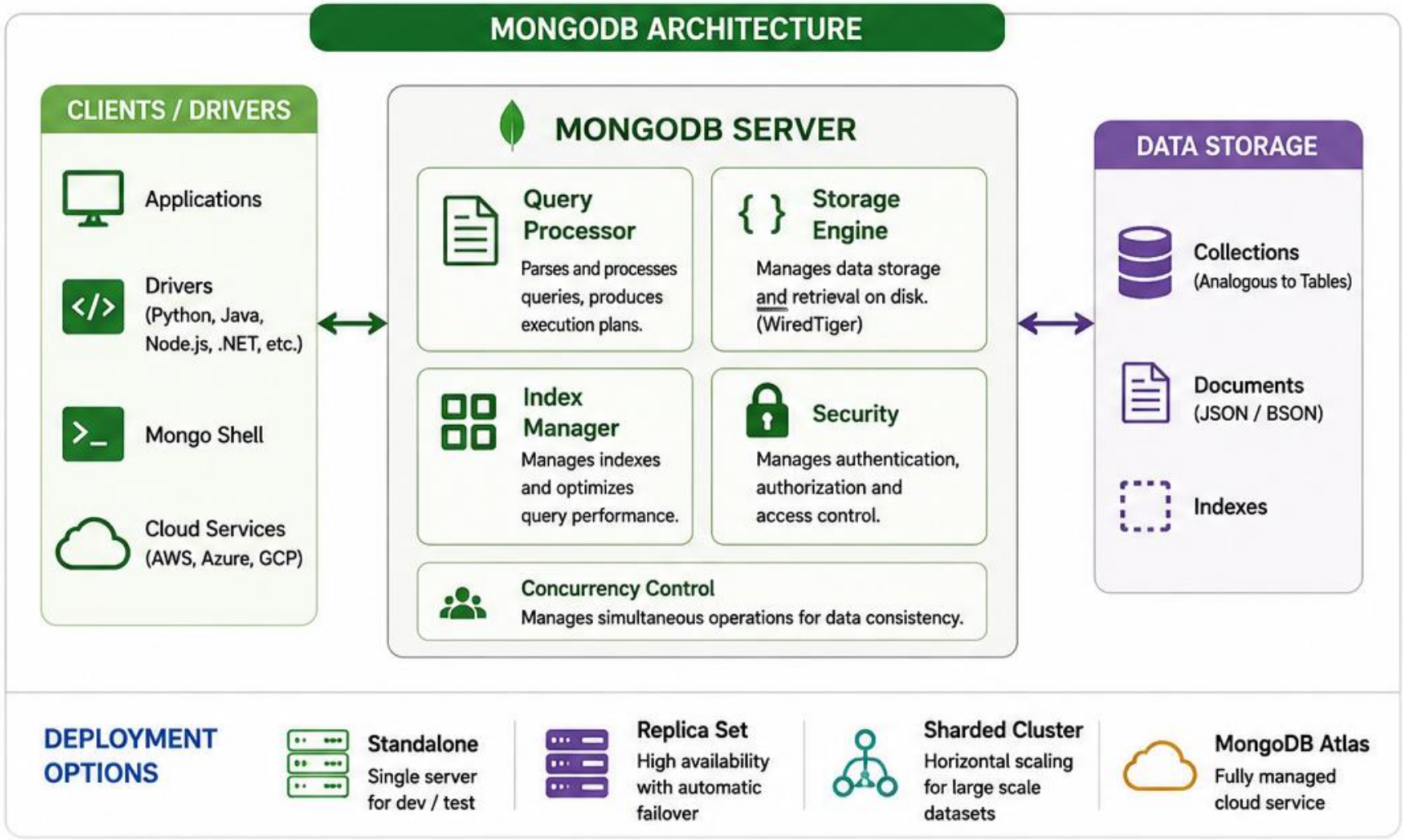
MONGODB ARCHITECTURE OVERVIEW

High-level view of MongoDB core architecture

MongoDB is a document-oriented, distributed database designed for high performance, high availability, and horizontal scalability. It follows a client-server architecture with flexible deployment options.

Why It Matters

- High Performance
- Horizontally Scalable
- High Availability
- Flexible Schema
- Cost Effective



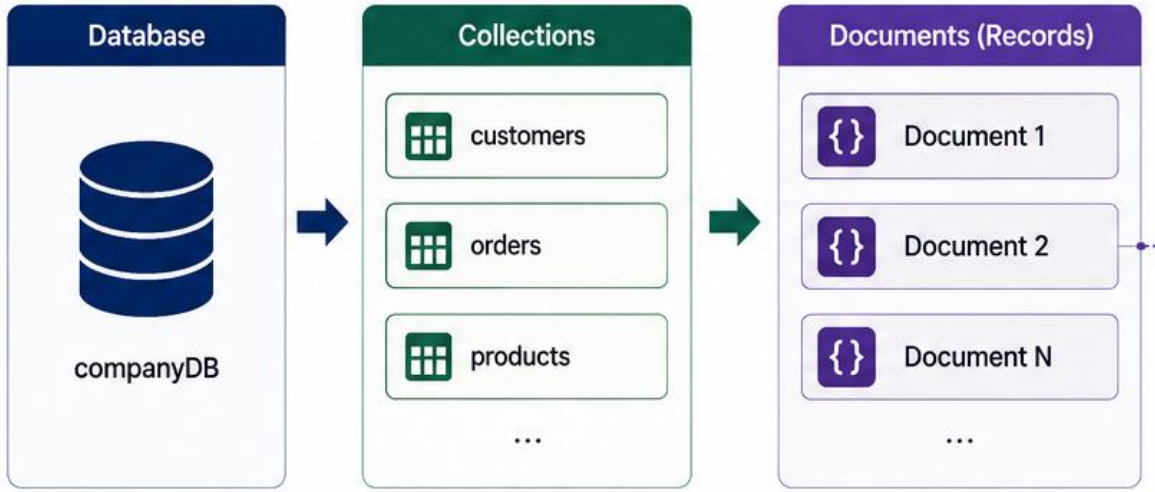
COLLECTIONS AND DOCUMENTS

The building blocks of MongoDB

MongoDB stores data in flexible, JSON-like documents organized into collections within databases. This structure provides schema flexibility, scalability, and performance for modern applications.

Collections organize documents, and documents store data in a flexible, JSON-like format—enabling agility, scalability, and high performance.

MONGO DB STRUCTURE



EXAMPLE DOCUMENT (JSON)

```
{
   "_id": ObjectId("65ecf3a8e1b2c8d9f8a7b123"),
   "name": "John Doe",
   "email": "john.doe@example.com",
   "phone": "9876543210",
   "address": {
      "street": "123 Business Ave",
      "city": "Bangalore",
      "state": "Karnataka",
      "zip": "560001"
   },
   "createdAt": ISODate("2024-03-10T10:15:30Z"),
   "isActive": true,
   "tags": ["premium", "enterprise"]
}
```

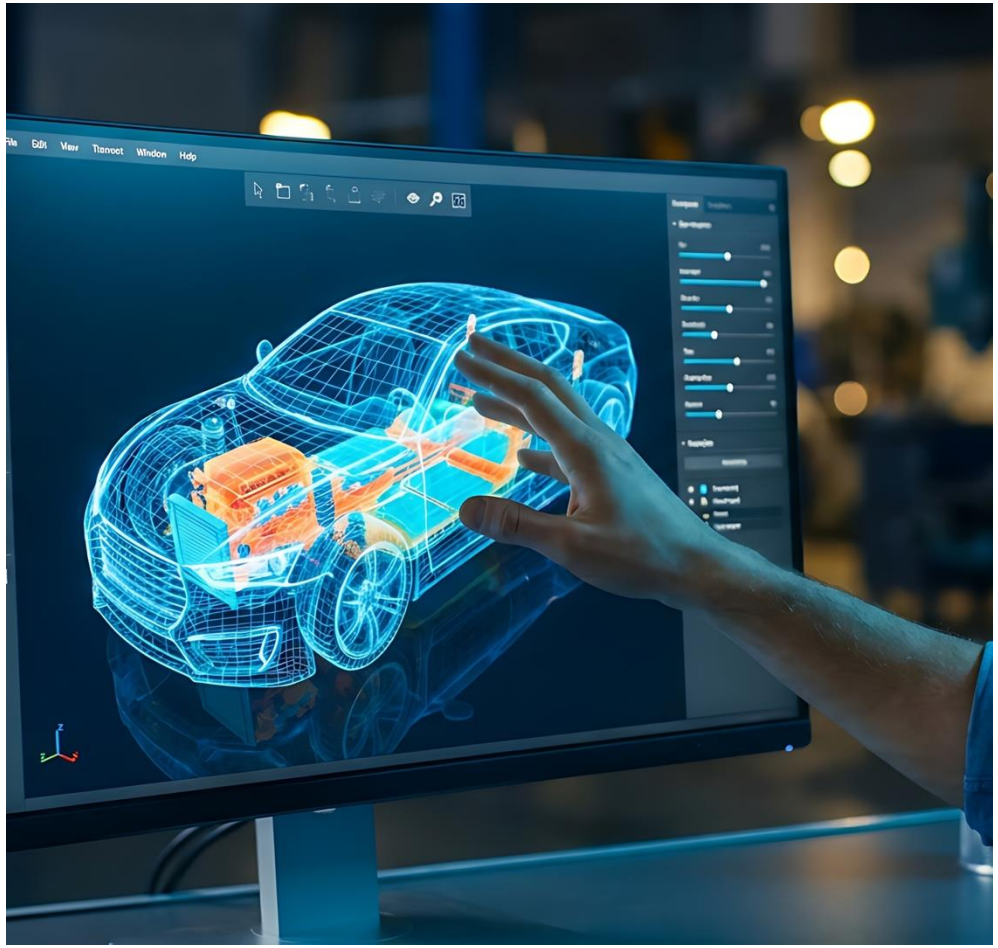
MONGODB DATA MODELING PRINCIPLES

Design flexible, scalable, high-performance data models

MongoDB uses a flexible document model. Good data modeling maximizes performance, ensures scalability, and keeps data easy to query and maintain.

Why It Matters

- Better Performance
- Horizontal Scalability
- Flexible Schema
- Simpler Queries
- Lower Operational Overhead



Core Data Modeling Principles

1. Model for Your Queries

Understand your application queries and access patterns. Design documents that support those queries efficiently.

Key Takeaway

Design around how data is read, not just how it is written.

2. Embed Related Data

Embed data that is frequently accessed together. Ideal for one-to-few relationships.

Key Takeaway

Embed for performance when relationships are tightly bound.

3. Reference When Appropriate

Use referencing for large or frequently changing data. Ideal for one-to-many or many-to-many relationships.

Key Takeaway

Reference when data is large, shared, or changes independently.

4. Avoid Unbounded Arrays

Keep arrays small and bounded. Large arrays can impact performance and index efficiency.

Key Takeaway

Unbounded arrays can hurt performance and scalability.

5. Normalize Where It Makes Sense

Apply normalization principles when needed for consistency. Avoid duplication of critical or reference data.

Key Takeaway

Denormalize for read performance, normalize for consistency.

6. Plan for Scalability

Choose the right shard key early. Distribute data evenly across shards. Avoid monotonically increasing fields.

Key Takeaway

Good models scale horizontally without hot spots.

Modeling Patterns in MongoDB

Embedded Document

Best for one-to-few relationships. Keeps related data together.

Referenced Document

Best for one-to-many or many-to-many relationships. Maintains data independence.

Bucket Pattern

Stores time-series or large data in buckets for efficient queries.

Hierarchy Pattern

Stores hierarchical data using parent references or materialized paths.

Outlier Pattern

Separates infrequent or large outlier data from main collection.

EMBEDDED DOCUMENTS VS REFERENCED DOCUMENTS

Two approaches to data modeling

MongoDB supports flexible data modeling — embed related data within a document or store it in separate collections and reference it. Choosing the right approach is key to building scalable, efficient applications.

Embedded Documents: When data is closely related, accessed together, and has a bounded size.

Referenced Documents: When data is large, grows independently, or is shared across multiple parent documents.

Embedded Documents

Related data is stored inside the parent document.

Single Collection

Collection: orders

```
{
  "_id": 101,
  "customer": "John Doe",
  "items": [
    { "product": "Laptop", "qty": 1, "price":
    },
    { "product": "Mouse", "qty": 2, "price": 25
    }
  ],
  "total": 1250
}
```

Advantages

- Fewer queries
- Faster read performance
- Atomic updates
- Ideal for one-to-few relationships

Considerations

- Document size grows
- Not suitable for frequently changing large arrays
- Data duplication risk

VS

Referenced Documents

Related data is stored in separate collections and linked using references.

Collection: orders

```
{
  "_id": 101,
  "customer": "John Doe",
  "itemIds": [
    ObjectId("a1"),
    ObjectId("a2")
  ],
  "total": 1250
}
```

Collection: items

```
{
  "_id": ObjectId("a1"),
  "product": "Laptop",
  "qty": 1,
  "price": 1200
}
```

```
{
  "_id": ObjectId("a2"),
  "product": "Mouse",
  "qty": 2,
  "price": 25
}
```

Advantages

- Supports large and growing data
- Avoids duplication
- Data can be shared across multiple parents
- Flexible updates

Considerations

- More queries (joins in application)
- Slightly slower reads
- Application must handle relationships

CRUD OPERATIONS OVERVIEW

The foundation of working with data in MongoDB

CRUD stands for Create, Read, Update, and Delete — the four core operations for data management in MongoDB, used in almost every application.

Why It Matters

- Essential for all applications
- Enables efficient data operations
- Supports scalable and flexible apps
- Helps maintain data integrity
- Improves developer productivity

THE 4 CRUD OPERATIONS

CREATE

Add new data

- Insert a single document
- Insert multiple documents
- Add new fields
- Default values and validation

```
db.collection.insertOne({...})
db.collection.insertMany([...])
```

READ

Retrieve data

- Query documents
- Filter with conditions
- Sort and project fields
- Pagination and limit

```
db.collection.find({...})
db.collection.findOne({...})
```

UPDATE

Modify existing data

- Update a single document
- Update multiple documents
- Update specific fields
- Use operators for changes

```
db.collection.updateOne({...})
db.collection.updateMany({...})
```

DELETE

Remove data

- Delete a single document
- Delete multiple documents
- Delete by condition
- Remove entire collection (use carefully)

```
db.collection.deleteOne({...})
db.collection.deleteMany({...})
```

MONGODB INDEXING FUNDAMENTALS

Speed up queries. Improve performance.

Indexes in MongoDB improve the speed of data retrieval operations. They create a data structure that stores a small portion of the collection's data in an easy-to-traverse form.

Why It Matters

- Faster Queries
- Better Performance
- Scalability
- Efficient Sorting
- Cost Savings

HOW INDEXING WORKS

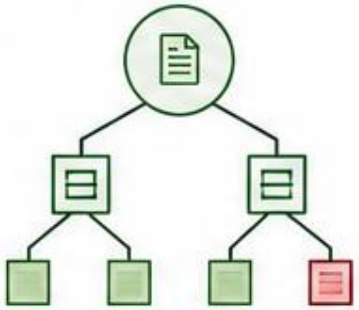
Without Index

Full Collection Scan

MongoDB scans every document to find matches.

With Index

Indexed Lookup



MongoDB usas the index to quickly locate matching documents.

Indexes trade disk space for speed.
More indexes improve reads but may slow down writes.

EXAMPLE

Create an Index

```
db.users.createIndex({ "email":  1 })
```

Query Using Index

```
db.users.find({ "email": "john@example.com" })
```

Explain Plan (Shows Index Usage)

```
db.users.find({ "email": "john@example.com" }
.explain("executionStats")
```

Key Points

- Indexes improve read performance.
- Write operations (insert/update/delete) are slower with many indexes.
- Choose the right index based on query patterns.
- Use `explain()` to verify index usage.

AGGREGATION FRAMEWORK OVERVIEW

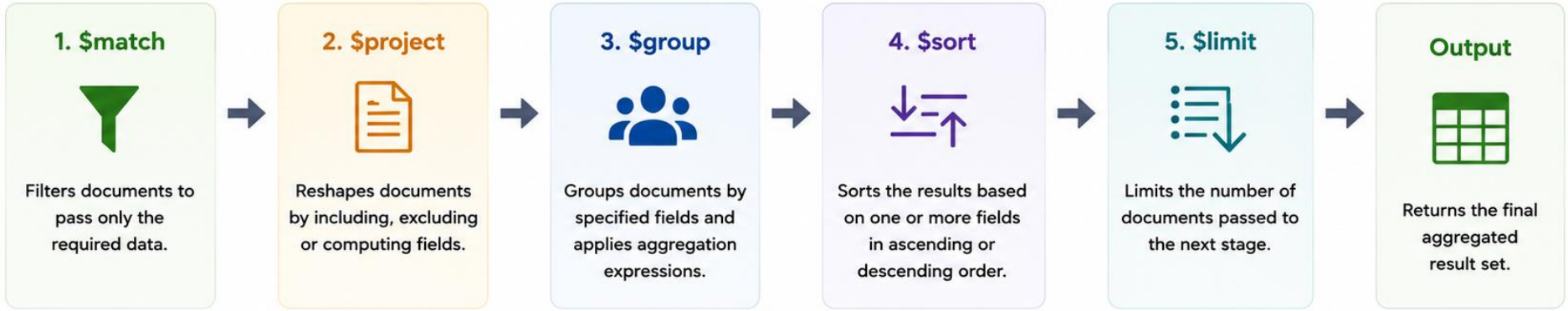
Powerful data processing and analytics in MongoDB

MongoDB Aggregation Framework allows you to process, transform and analyze data using a pipeline of stages. It helps perform complex queries and generate meaningful insights efficiently.

Why It Matters

- Powerful Analytics
- Filter and Transform Data Efficiently
- Reduce Data Transfer
- Improve Query Performance
- Lower Operational Cost

How It Works



Common Aggregation Stages

Stage	Purpose
$match	Filters documents
$project	Reshapes documents
$group	Groups and aggregates
$sort	Sorts documents
$limit / $skip	Limits or skips documents
$unwind	Deconstructs arrays
$lookup	Joins with another collection
$count	Counts the number of documents

Example Pipeline

```
[
  { $match: { status: "A" } },
  { $project: { item: 1, category: 1, price: 1, year: 1 } },
  { $group: { _id: "$category", totalSales: { $sum: "$price" } }},
  { $sort: { totalSales: -1 } },
  { $limit: 5 }
]
```

The pipeline processes data stage by stage and returns aggregated results based on your business logic.

Use Cases

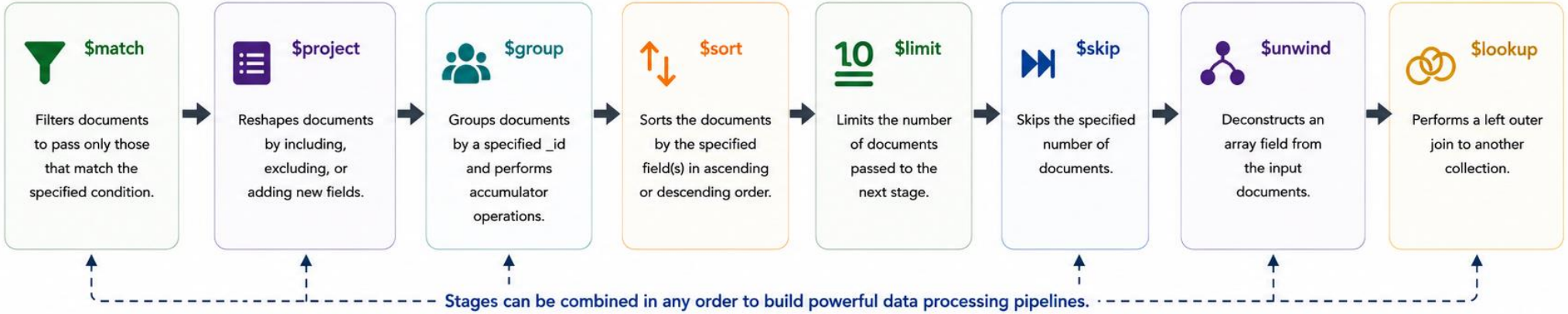
- Sales analytics and reporting
- E-commerce insights
- Customer segmentation
- Time-series analysis
- Dashboards and summaries

AGGREGATION PIPELINE STAGES

Transform data. Extract insights.

The MongoDB Aggregation Framework provides a powerful way to process and transform data through a pipeline of stages. Each stage takes the output of the previous stage and passes it to the next._x000B__x000B_Aggregation pipeline stages work together to transform raw data into meaningful insights. Choose the right stages and order to build efficient and powerful data workflows.

Common Aggregation Pipeline Stages



Example: Sales Analysis Pipeline

Pipeline Query Example

```
db.orders.aggregate([
  { $match: { orderDate: { $gte: ISODate("2024-01-01") } } },
  { $group: { _id: "$custId", totalSales: { $sum: "$amount" } } },
  { $sort: { totalSales: -1 } },
  { $limit: 10 }
])
```

Sample Output

_id (custId)	totalSales
C001	125,430
C043	98,210
C018	76,540
...	...

Top 10 customers

MONGODB ATLAS OVERVIEW

The leading cloud-native MongoDB service

MongoDB Atlas is a fully managed cloud database that simplifies deployment, operations, and scaling — so teams can focus on building applications instead of managing infrastructure.

Key Capabilities

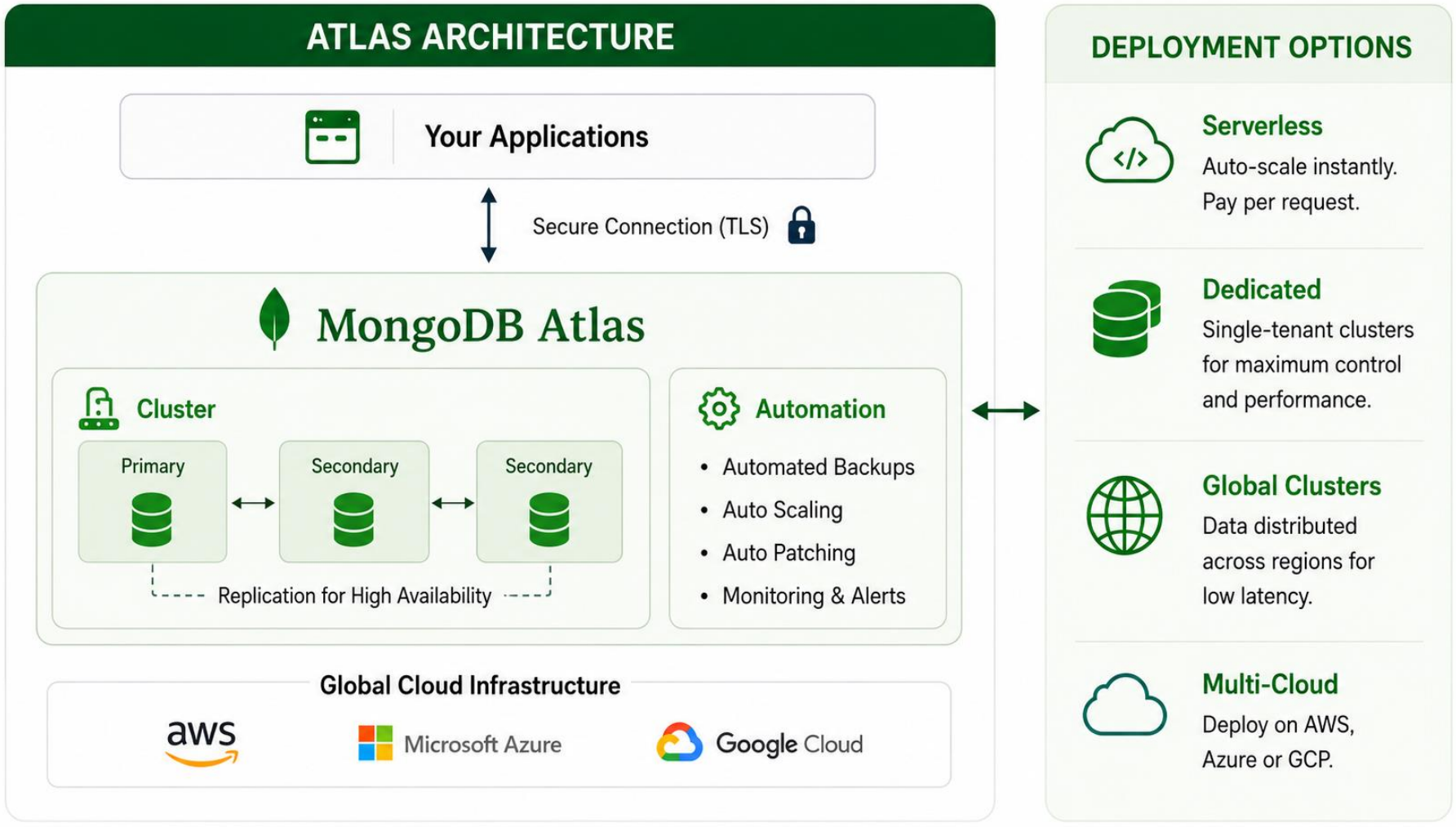
Intuitive UI & APIs

Enterprise Security

Automated Backups & Point-in-Time Restore

Performance Monitoring

Integrations



MONGODB USE CASES IN MODERN ENTERPRISES

MongoDB's document model, rich query capabilities, and horizontal scalability make it ideal for a wide range of modern enterprise applications and data-driven initiatives

Why It Matters

- Accelerate Time to Market
- Scale Effortlessly as You Grow
- Increase System Resilience
- Optimize Costs and Resources
- Enable Innovation with Data

KEY MONGODB USE CASES

1. Customer 360 & Personalization

- Unite customer data from multiple touchpoints
- Real-time profiles and behavioral insights
- Power personalized experiences

Includes:
Retail, Banking, Telecom, Healthcare

2. Content Management & Digital Experiences

- Manage dynamic and unstructured content
- Support high-scale websites and mobile apps
- Real-time content updates and delivery

Includes:
Media, Publishing, E-commerce, Entertainment

3. Real-time Analytics & Data Platforms

- Ingest and analyze large volumes of real-time data
- Enable advanced analytics and insights
- Support operational and analytical workloads

Includes:
Finance, IoT, Logistics, Manufacturing

4. IoT & Operational Data Management

- Store and process high-velocity IoT data
- Handle diverse device data formats
- Enable real-time monitoring and alerts

Includes:
Manufacturing, Energy, Smart Cities, Logistics

5. Application Modernization

- Modernize legacy applications with minimal changes
- Flexible data model adapts to new requirements
- Reduce time and cost of transformation

Includes:
Government, Insurance, Healthcare, BFSI

6. Fraud Detection & Risk Management

- Analyze transactions and user behavior in real time
- Detect anomalies and reduce fraud
- Improve decisioning with real-time insights

Includes:
Banking, Fintech, E-commerce, Payments

MODULE 2 — INFORMATICA ETL FUNDAMENTALS

INTRODUCTION TO ETL

Extract. Transform. Load.

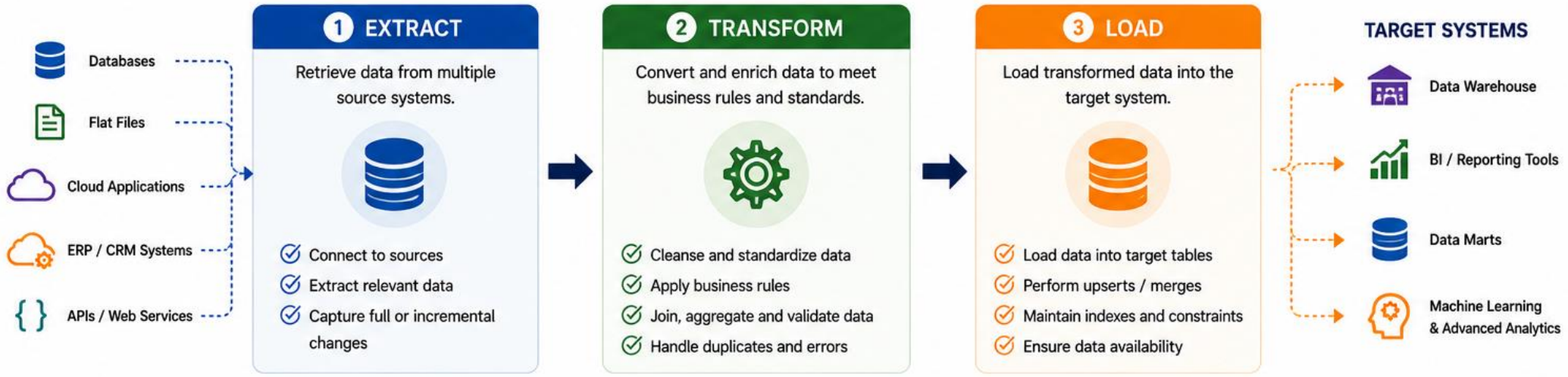
ETL is a data integration process used to collect data from multiple sources, transform it into a consistent and meaningful format, and load it into a target system for analytics and reporting.

ETL is the foundation of modern data warehouses and business intelligence solutions.

Why ETL Matters

- Integrates Data
- Improves Data Quality
- Enables Better Insights
- Supports Scalability
- Ensures Governance

THE ETL PROCESS



KEY CONCEPTS

Batch Processing
Data is processed in scheduled intervals.

Incremental Load
Only new or changed data is extracted and loaded.

Data Quality
Ensures data is accurate, complete, consistent and valid.

Metadata
Stores information about data structures, mappings and processes.

Logging & Monitoring
Tracks executions, errors and performance for reliability.

ETL VS ELT

Choosing the Right Approach for Your Data Platform

Both ETL and ELT move data from source to destination, but the transformation happens at different stages.

Choose the approach that best fits your data, tools and business goals.

ETL transforms data before loading; ELT transforms after loading.

ETL (Extract, Transform, Load)

Transform data before loading into the target system

1. Extract
Data is extracted from source systems

2. Transform
Data is cleaned, filtered and transformed in staging/ETL tools

3. Load
Transformed data is loaded into the target data warehouse

Best Suited For

- Structured data with well-defined schemas
- Business rules applied before loading
- Data quality must be enforced early
- Traditional data warehouses

KEY DIFFERENCES

- Transformation Timing
- Processing Location
- Performance
- Cost
- Flexibility
- Use Cases

ELT (Extract, Load, Transform)

Load data first, then transform inside the target system

1. Extract
Data is extracted from source systems

2. Load
Raw data is loaded into the target (Data Lake / Warehouse)

3. Transform
Transformations are performed inside the target system using native compute

Best Suited For

- Large volumes of raw or semi-structured data
- Advanced analytics and data exploration
- Flexible schemas and evolving data
- Cloud data platforms and data lakes

WHEN TO USE WHICH?

ETL

- When data needs heavy cleansing and business rule validation upfront
- When the target system has limited transformation capabilities
- When data quality and consistency are critical before loading
- When working with traditional data warehouses

ELT

- When dealing with large volume, variety and velocity of data
- When using cloud data warehouses or data lakes
- When schema is flexible and data needs to be explored
- When transformations are iterative and experimental

ENTERPRISE DATA INTEGRATION CHALLENGES

Why Integration Is Harder Than Ever

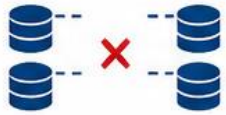
Enterprises must integrate data from diverse sources, formats, platforms, and environments. Siloed systems, growing data volumes, and business expectations create significant complexity and risk.

Business Impact

- Higher operational costs
- Delayed time to market
- Poor decision making
- Increased risk and compliance issues
- Low customer satisfaction

KEY ENTERPRISE DATA INTEGRATION CHALLENGES

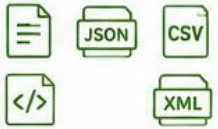
SILOED SYSTEMS AND DATA



- Data spread across multiple systems
- Lack of standardization
- Limited data visibility

Makes it hard to get a single, trusted view

DIVERSE SOURCES AND FORMATS



- Structured, semi-structured and unstructured data
- Multiple file formats
- Constantly changing source systems

Complex to connect, parse and transform

DATA VOLUME AND VELOCITY

- Exploding data growth
- Real-time and batch processing needs
- High throughput requirements

Challenges performance and scalability

DATA QUALITY AND CONSISTENCY

- Inconsistent, incomplete or duplicate data
- Lack of data validation
- Poor data governance

Leads to inaccurrate insights and poor decisions

INTEGRATION COMPLEXITY

- Complex mappings and transformations
- Business rule variations
- Hard to maintain workflows

Slows delivery and increases maintenance

SECURITY, PRIVACY AND COMPLIANCE

- Protecting sensitive data across systems
- Access control
- Regulatory compliance (GDPR, HIPAA, etc.)

Risk of breaches, fines and data misuse

INFORMATICA PLATFORM OVERVIEW

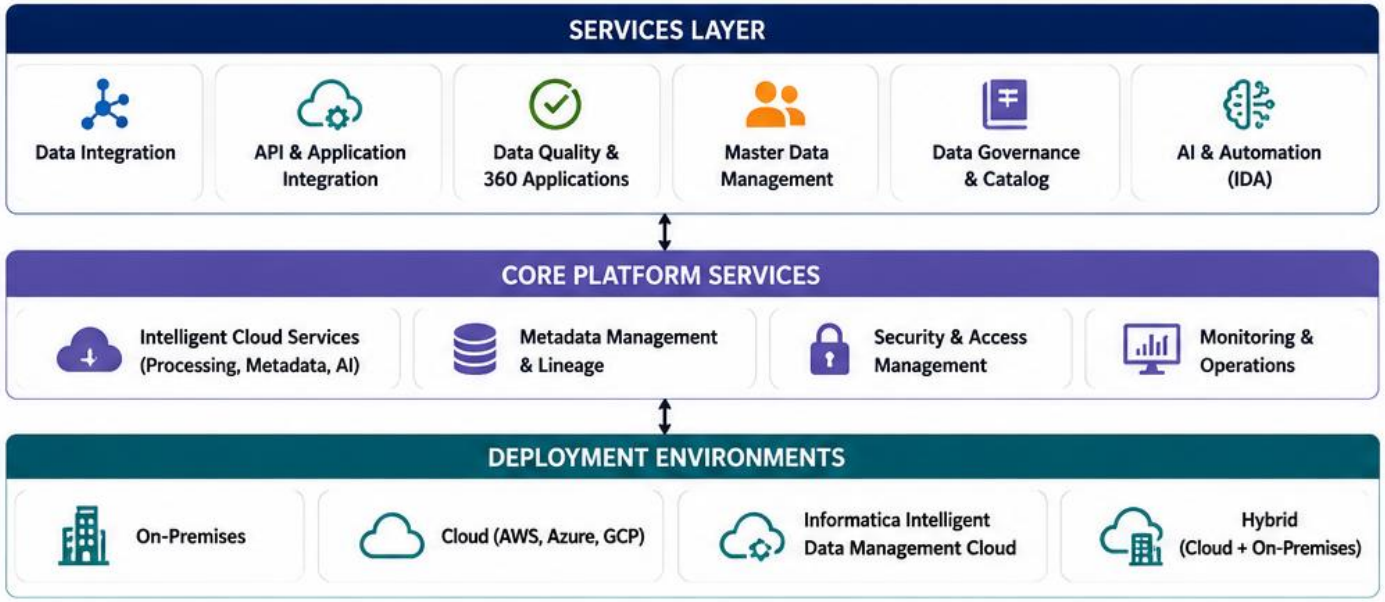
Enterprise-Grade Data Integration Platform

Informatica is a leading data management platform that helps organizations integrate, manage, and govern data across cloud, on-premises, and hybrid environments.

Why It Matters

- Accelerate Data Integration and Delivery
- Ensure Data Quality and Consistency
- Enable Hybrid and Cloud Flexibility
- Strengthen Data Governance and Security
- Improve Efficiency and Reduce Time to Value

INFORMATICA PLATFORM COMPONENTS



INFORMATICA CLIENT TOOLS

PC — PowerCenter Designer
Design mappings, workflows, and reusable objects.

PM — PowerCenter Workflow Manager
Monitor, schedule, and manage workflows and sessions.

DT — Data Director
Manage metadata, security, and user permissions.

CA — Cloud Application
Web-based UI for monitoring and administration.

IDMC — Informatica IDMC Console
Manage cloud services, connections, and assets.

KEY CAPABILITIES

End-to-End Data Integration
Connect, transform, and deliver data from any source to any target.

High Performance & Scalability
Parallel processing and in-memory engine for faster data processing at scale.

Enterprise Security
Role-based access, encryption, and compliance with industry standards.

Intelligent Automation
AI-driven insights, recommendations, and automation with IDA.

Extensibility & Flexibility
Open APIs, connectors, and support for modern data ecosystems.

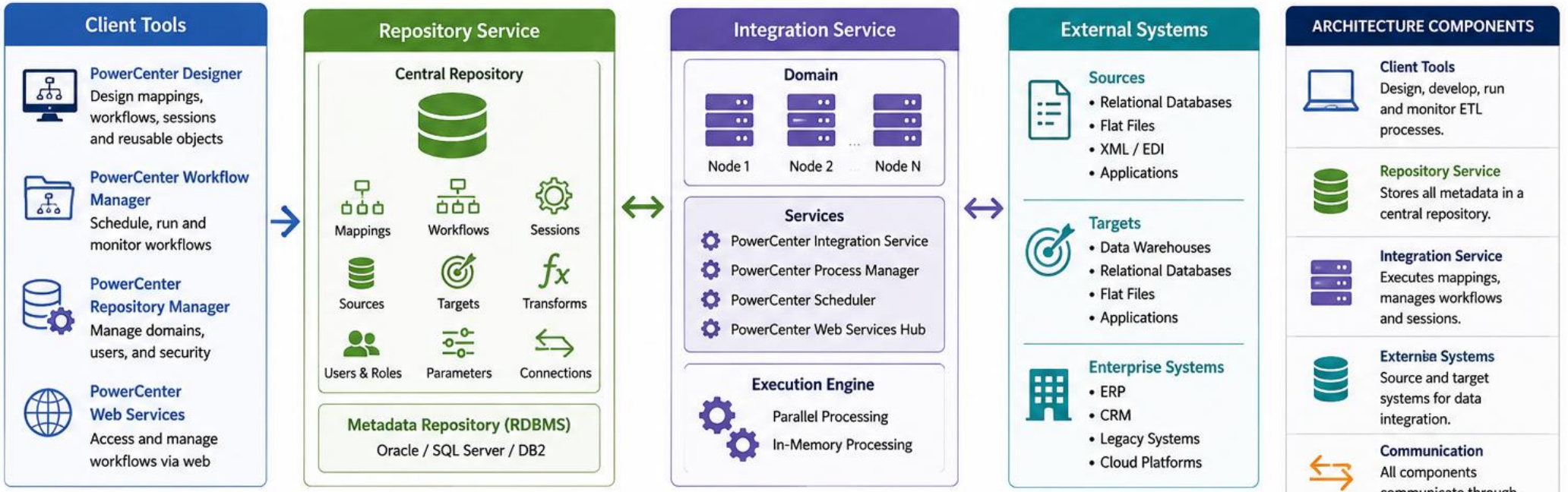
INFORMATICA POWERCENTER ARCHITECTURE

High-Performance ETL Platform for Enterprise Data Integration

Informatica PowerCenter is a scalable, robust, and metadata-driven ETL platform that enables organizations to integrate, transform, and manage data across diverse systems.

It provides a robust, scalable, and metadata-driven
foundation to integrate data across diverse enterprise environments with high performance and reliability.

POWERCENTER ARCHITECTURE OVERVIEW



KEY ARCHITECTURE FEATURES

Metadata-Driven
Centralized metadata ensures consistency and reusability.

Scalability
Scale resources by adding nodes to the domain.

High Availability
Domain and repository can be configured for high availability.

Performance
Parallel processing, pushdown optimization and in-memory engine.

Security
Role-based access, encryption and secure communication.

Extensibility
Reusable components, web services and custom plug-ins.

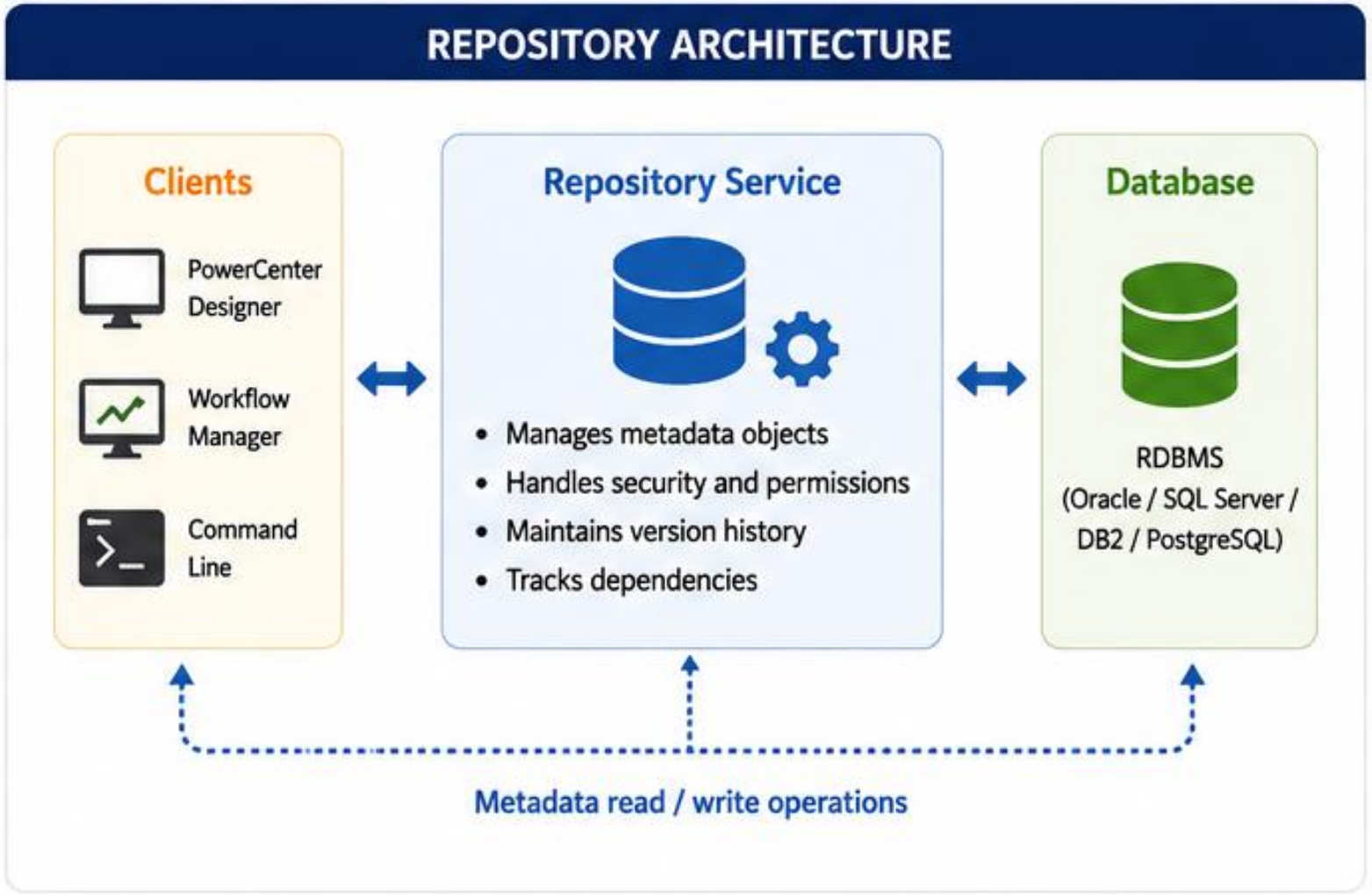
REPOSITORY SERVICE

The Central Metadata Store of Informatica

The Repository Service is a multi-user, centralized metadata repository that stores and manages all metadata objects used by Informatica PowerCenter. It acts as the backbone of the entire platform, enabling sharing, consistency, security, and reusability.

Why It Matters

- Centralizes all metadata in a single repository
- Ensures data consistency and classification
- Enables sharing and collaboration across teams
- Supports security and access control
- Improves productivity and governance



INTEGRATION SERVICE

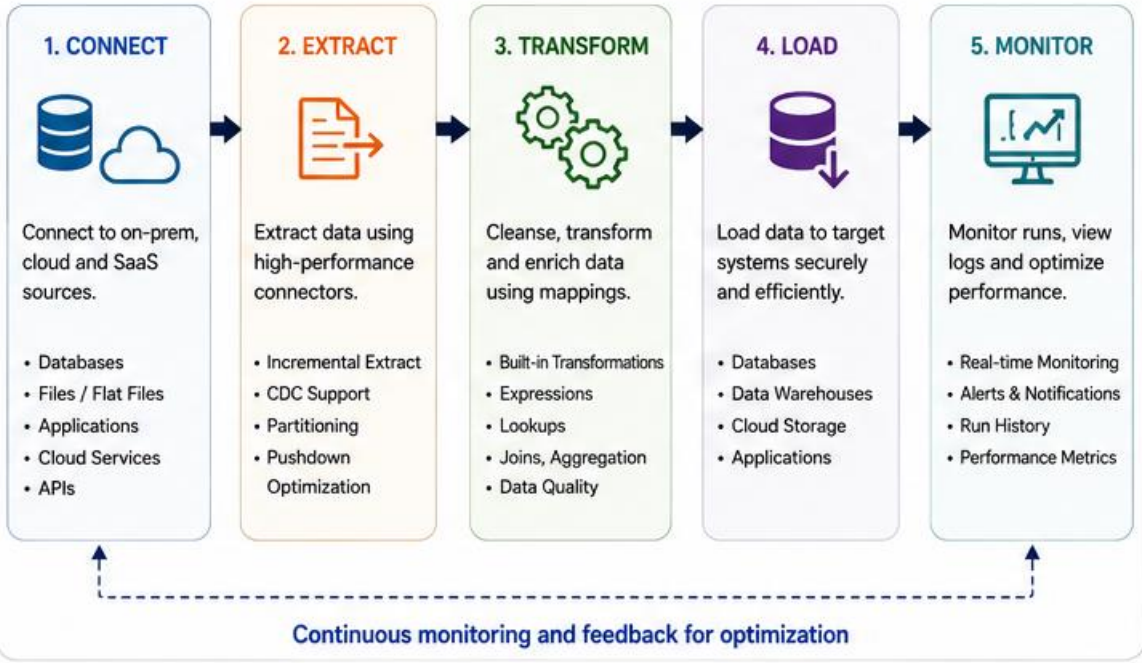
The Cloud-Native ETL & Data Integration Engine

Informatica Integration Service (IICS) is a fully managed, cloud-native data integration service that enables you to build, run, and manage secure, scalable ETL and ELT processes in the cloud.

Why Integration Service?

- Cloud-Native
- Scalable
- Secure
- High Performance
- Cost Efficient

HOW INTEGRATION SERVICE WORKS



COMMON USE CASES

- Cloud Data Integration
- Data Warehouse Loading
- Data Migration to Cloud
- Analytics & Reporting
- Change Data Capture (CDC)
- Master Data Management
- Application Integration

KEY FEATURES

Fully Managed Service
Informatica manages infrastructure, patching, scaling and availability.

Elastic Environment
Compute resources scale automatically based on workload demand.

Enterprise Security
SSO, RBAC, encryption, VPC peering and network isolation.

Rich Connectors
Prebuilt connectors for 1000+ sources and targets.

Advanced Monitoring
Real-time dashboards, insights and alerting.

Scheduling & Orchestration
Schedule jobs, use triggers and orchestrate complex workflows.

INFORMATICA MAPPING FUNDAMENTALS

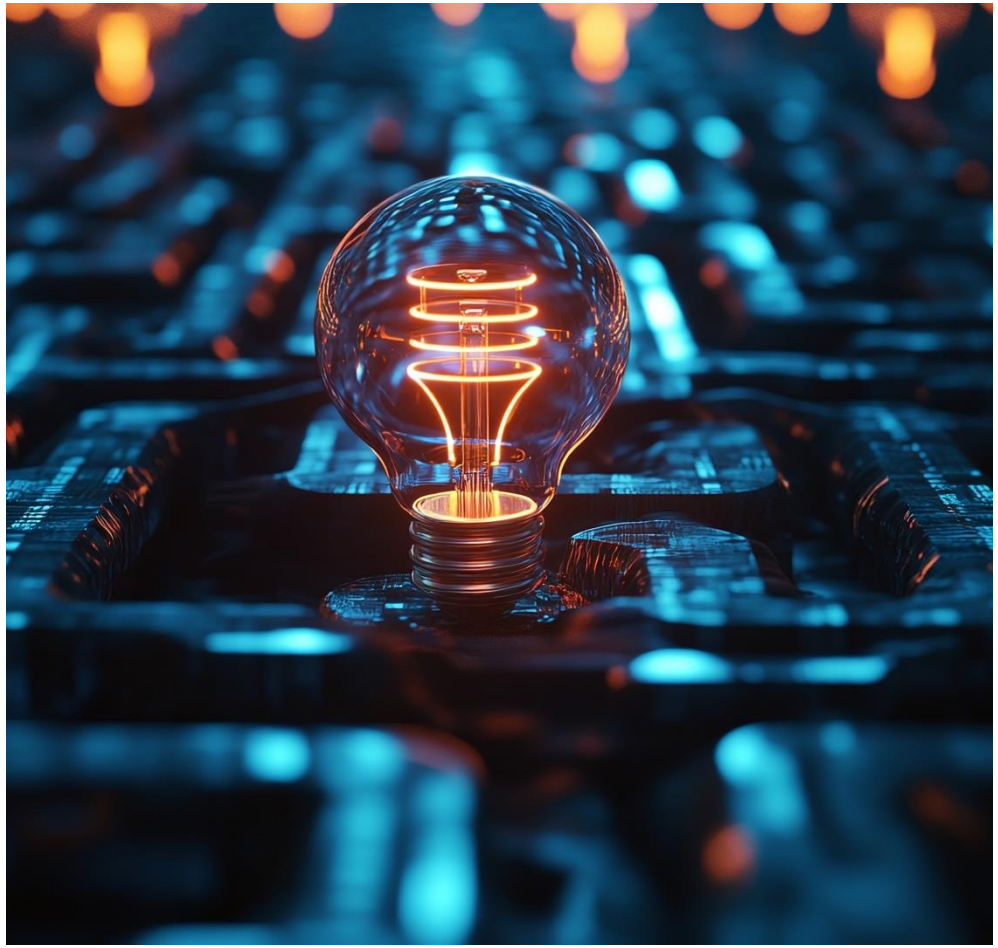
The Heart of ETL Development

A Mapping in Informatica defines how data flows from one or more sources through transformations to one or more targets.

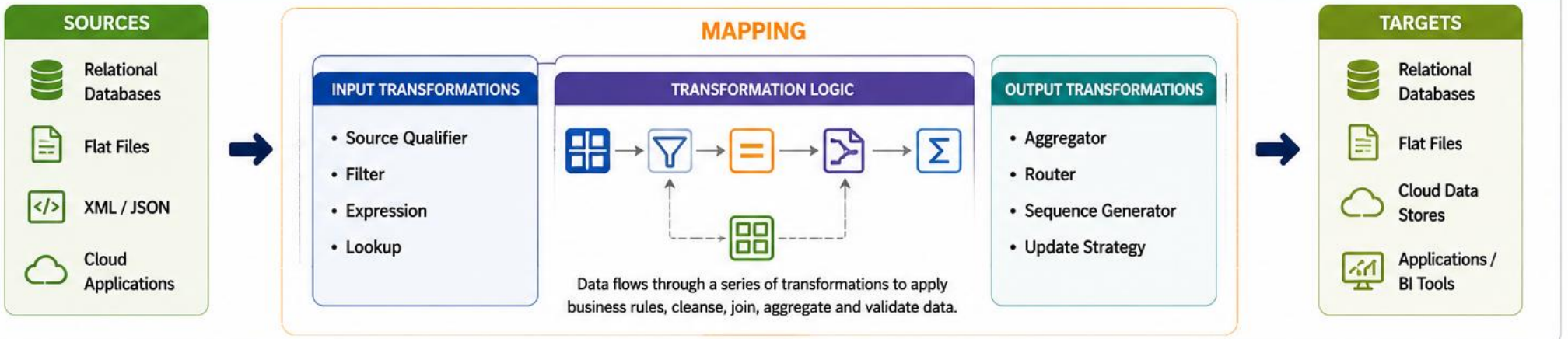
It performs data extraction, transformation, and loading

Why It Matters

- Central to Data Integration
- Enables Complex Transformations
- Ensures Data Quality
- Improves ETL Performance
- Supports Reusability and Scalability
- logic.



MAPPING ARCHITECTURE



KEY CONCEPTS

Data Flow
Data is extracted from sources and flows through transformations to targets.

Transformations
Pre-built components used to cleanse, transform, validate, and enrich data.

Ports
Each transformation has input and output ports to pass data to next step.

Links
Links connect the output port of one transformation to the input port of another.

Reusable
Mappings can be reused as Mapplets for consistency and productivity.

BEST PRACTICES

- Push transformations down to the database when possible.
- Use meaningful names for transformations and ports.
- Reuse Mapplets to improve consistency and reduce effort.
- Apply filters early to reduce data volume.
- Validate data at each step to ensure data quality.
- Monitor mapping performance and optimize regularly.

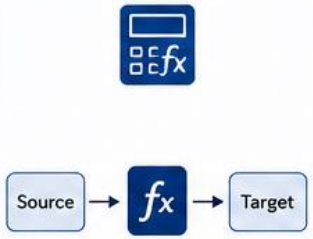
CORE TRANSFORMATIONS OVERVIEW

Transformations enable you to manipulate, filter, route and combine data to meet business requirements.

- Expression
- Filter
- Router
- Lookup
- Joiner
- Aggregator

These core transformations are the building blocks of effective ETL mappings. Mastering them enables you to cleanse, enrich, combine and summarize data to deliver trusted business results.

Expression



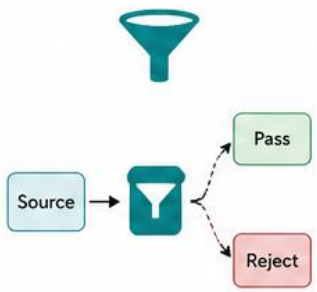
Purpose

Creates new fields and performs calculations or data manipulations using expressions.

Common Use Cases

- Derive new columns
- Data type conversions
- String/date/number functions
- Conditional logic

Filter



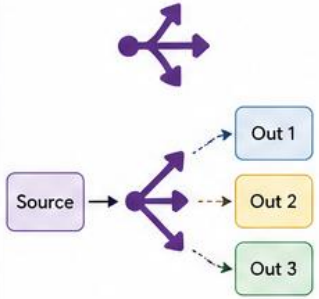
Purpose

Filters rows based on conditions and separates them into Pass or Reject groups.

Common Use Cases

- Data quality checks
- Remove invalid records
- Business rule filtering
- Deduplication

Router



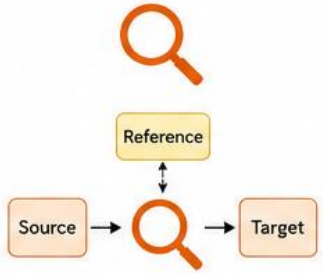
Purpose

Routes rows to one of multiple output groups based on specified conditions.

Common Use Cases

- Segregate data by type
- Route by region or business unit
- Handle multiple scenarios in a single mapping

Lookup



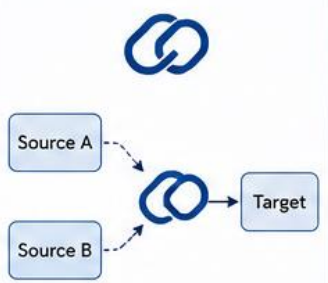
Purpose

Looks up matching rows in a reference dataset and returns related values.

Common Use Cases

- Enrich data
- Code to description mapping
- Surrogate key lookup
- Default value handling

Joiner



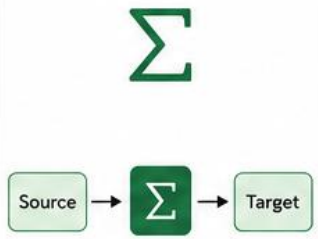
Purpose

Combines rows from two or more sources based on matching columns (Inner, Outer, Left, Right joins).

Common Use Cases

- Combine master and transaction data
- Consolidate datasets
- Create denormalized views

Aggregator



Purpose

Groups rows and performs summary calculations.

Common Use Cases

- SUM, COUNT, AVG, MIN, MAX
- Group by customer, product, date, etc.
- Generate summary reports

WORKFLOW DEVELOPMENT

A Workflow in Informatica PowerCenter defines the sequence of tasks to execute and the dependencies between them. It ensures reliable, repeatable and manageable execution of ETL processes.

- Automates complex ETL processes end-to-end
- Improves productivity and reduces manual effort
- Ensures reliability with error handling and recovery
- Enhances visibility and monitoring of process execution
- Enables reusability and standardization across teams

WHAT IS A WORKFLOW?

A Workflow is a container that organizes and controls the execution of tasks in a logical sequence with defined dependencies.

KEY CHARACTERISTICS

Sequencing
Tasks are executed in a defined order.

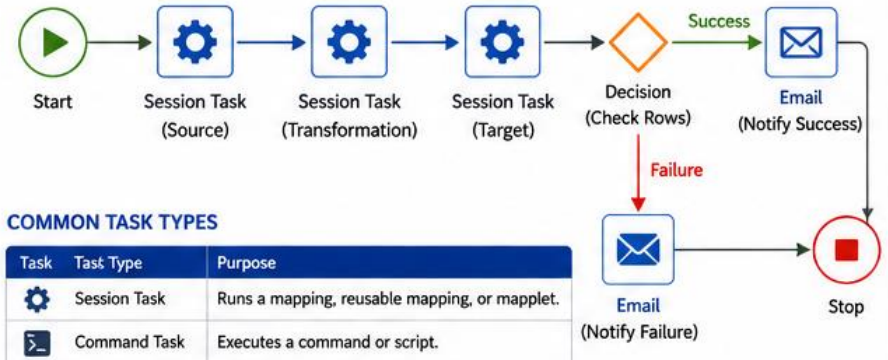
Dependencies
Defines relationships and execution rules between tasks.

Error Handling
Configure what happens on success or failure.

Reusability
Workflows can be reused as tasks in other workflows.

Monitoring
Track execution status, logs and performance.

HOW IT WORKS



COMMON TASK TYPES

Task	Task Type	Purpose
	Session Task	Runs a mapping, reusable mapping, or mapplet.
	Command Task	Executes a command or script.
	Workflow Task	Invokes another workflow.
	Email Task	Sends notification emails.
	Decision Task	Evaluates a condition and routes the flow.
	Wait Task	Pauses workflow execution for a specified time.

BEST PRACTICES

Plan Before You Build
Define the business process and task dependencies clearly.

Use Modular Design
Break complex workflows into smaller, reusable workflows.

Implement Error Handling
Always handle success and failure scenarios explicitly.

Monitor and Alert
Enable logging and notifications for proactive issue resolution.

Follow Naming Standards
Use consistent naming for workflows, tasks and variables.

WORKFLOW DEVELOPMENT LIFECYCLE

1. Requirements
Understand business needs and define process flow.

2. Design
Design workflow structure, tasks and dependencies.

3. Build
Create workflows, configure tasks and define logic.

4. Test
Validate workflow logic, data flow and error scenarios.

5. Deploy
Deploy to target environment and schedule execution.

6. Monitor & Maintain
Monitor performance, handle issues and optimize.

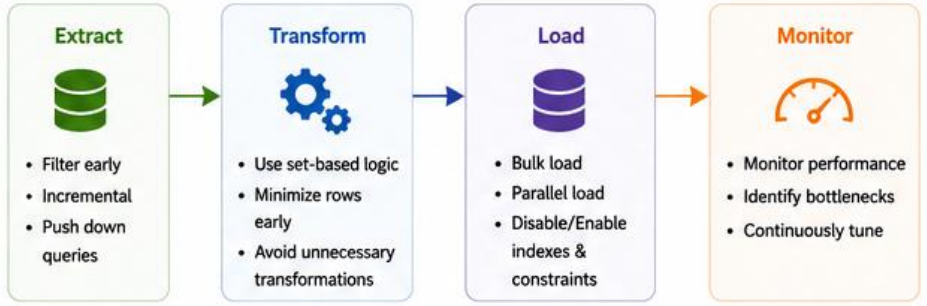
ETL PERFORMANCE OPTIMIZATION

Optimizing ETL performance ensures faster load times, efficient resource utilization, lower infrastructure costs and reliable SLAs. It requires the right design choices, tuning parameters and continuous monitoring.

Key Benefits

- Faster Load Times
- Higher Throughput
- Efficient Resource Utilization
- Lower Infrastructure Costs
- Reliable SLAs & Better Data Quality

OPTIMIZATION IN ETL FLOW



KEY PERFORMANCE METRICS

Load Time
Total time taken for the workflow

Throughput
Rows processed per second

CPU & Memory Utilization
Resource usage
Resource usage by the session

Wait Time
I/O, Locks and Network waits

Error Rate
Number of errors/rejects

COMMON BOTTLENECKS AND SOLUTIONS

Bottleneck	Impact	Solution
Large Data Volume	Long load times, high resource usage	Filter early, incremental load, partitioning, parallel load
Inefficient Lookups	Slow transformations	Use lookup cache, reference qualifier, reduce lookups
Unoptimized Mappings	High CPU and memory usage	Use set-based logic, avoid unnecessary expressions
Slow Target Load	Inserts/updates are slow	Use bulk load, disable indexes, optimize commit size
Infrastructure Limits	Resource contention, long waits	Increase CPU/Memory, use SSD, tune DB parameters

BEST PRACTICES

- Extract only what you need, when you need.
- Filter early and reduce data volume as soon as possible.
- Use appropriate join strategies and avoid Cartesian joins.
- Use lookup caches and reference tables efficiently.
- Use parallel sessions and partitioning for large data volumes.
- Use bulk load and commit size tuning.
- Disable indexes, constraints and triggers during load; rebuild after load.
- Monitor, measure and tune continuously.

PERFORMANCE OPTIMIZATION TIPS

Push Filters to Source
Reduce data as early as possible.

Parallelism
Use multiple sessions and partitions.

Bulk Operations
Use bulk load and set-based operations.

Monitor & Measure
Use performance counters and logs.

Iterate & Improve
Continuously analyze and fine-tune.

ETL DEVELOPMENT BEST PRACTICES

Following best practices during ETL development ensures data quality, improves performance, simplifies maintenance and supports scalability of enterprise data solutions.

Good ETL is not just about moving data; it is about delivering accurate, complete, and timely information with efficiency, reliability, and trust.

1. DESIGN & DEVELOPMENT

Understand business requirements
Clearly define scope, data flows, rules and success criteria.

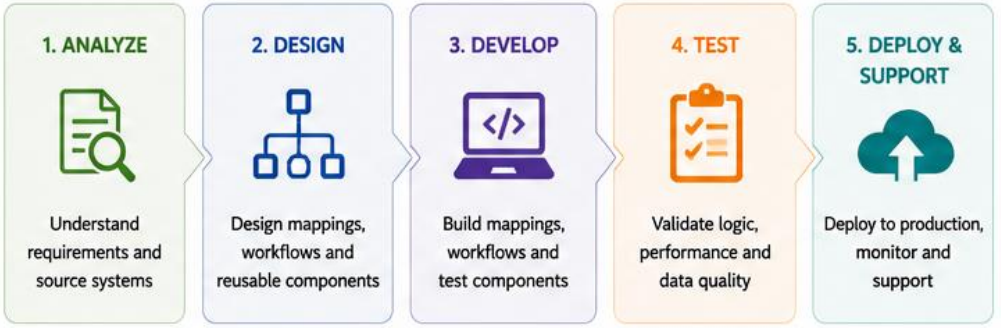
Design for clarity and simplicity
Keep mappings modular, reusable and easy to understand.

Use standards and naming conventions
Follow consistent naming for folders, sessions, mappings and variables.

Document everything
Maintain design documents, data dictionaries and mapping logic.

Perform impact analysis
Analyze impact of source changes on downstream mappings and targets.

ETL DEVELOPMENT LIFECYCLE



2. PERFORMANCE & OPTIMIZATION

- Optimize source queries and use database pushdown where possible.
- Use appropriate transformation strategies and connectivity.
- Use bulk loading and target load strategies for better performance.
- Lookups: use static lookups wherever possible.
- Tune mappings, sessions and workflows based on metrics.

3. DATA QUALITY & RELIABILITY

Validate data at every stage
Implement data validation rules, constraints and checks.

Handle exceptions properly
Capture, log and route bad data for resolution.

Implement auditing and logging
Maintain detailed logs for audit and troubleshooting.

Ensure data consistency
Use transactions, commits and rollback strategies.

Reconcile data
Perform source-to-target reconciliation to ensure completeness and accuracy.

4. MAINTENANCE & SUPPORT

- Schedule regular maintenance and cleanup of unused objects.
- Monitor jobs and set up proactive alerts for failures and thresholds.
- Version control mappings and promote through environments.
- Knowledge transfer and cross-training for better team support.
- Review and refactor mappings periodically to remove technical debt.

5. GOVERNANCE & SECURITY

- Implement role-based access controls.
- Protect sensitive data (encryption, masking).
- Follow change management process.
- Ensure compliance with data policies and regulations.

KEY TAKEAWAY

Following ETL best practices helps build robust, high-performance and reliable integrations that deliver **trusted data** for better business decisions.

MODULE 3 — DATA MIGRATION USE CASES

INTRODUCTION TO DATA MIGRATION

Move Your Data. Empower Your Business.

Data migration is the process of extracting data from a source system, transforming it as needed, and loading it into a target system.

It ensures data is accurate, consistent, and available in the right place to support business operations and growth.

- Improve system performance and scalability
- Support business transformation and modernization
- Migrate to cloud for agility and cost savings
- Ensure data availability, security and compliance
- Enable better analytics and decision making.

WHAT IS DATA MIGRATION?

Extract
Retrieve data from source system

Transform
Cleanse, validate and convert data

Load
Load data into target system

Goal: Move data accurately and efficiently with minimal disruption to business.

DATA MIGRATION PROCESS

1. **Assess**
   Evaluate source and target systems, data volume, quality and dependencies.
2. **Plan**
   Define migration strategy, scope, timeline, resources and success criteria.
3. **Extract & Transform**
   Extract data and cleanse, map, standardize and transform it as required.
4. **Load**
   Load data into target system and ensure referential integrity.
5. **Validate & Monitor**
   Validate data accuracy and completeness. Monitor and fix issues.

COMMON MIGRATION TYPES

Storage Migration
Move data between storage systems or formats.

Database Migration
Migrate between different databases or versions.

Application Migration
Move data as part of application modernization.

Cloud Migration
Move data from on-premises to cloud or between clouds.

KEY CHALLENGES

- Data quality issues and inconsistencies
- Large data volumes and complexity
- Heterogeneous systems and formats
- Downtime and business disruption
- Security, compliance and regulatory risks

BEST PRACTICES

- Profile and cleanse data before migration.
- Define clear data mapping and transformation rules.
- Perform multiple test migrations and validations.
- Monitor performance and address issues early.
- Ensure data security and compliance throughout.

SUCCESS FACTORS

- Strong planning and stakeholder alignment
- Accurate data mapping and governance
- Right tools and skilled team
- Thorough testing and validation
- Continuous monitoring and support

BUSINESS DRIVERS FOR DATA MIGRATION

Move Data. Unlock Value. Drive Business Outcomes.

Data migration is more than a technical initiative—it's a strategic enabler that helps organizations modernize, optimize and stay competitive.

Why It Matters

- Improves agility and responsiveness
- Enhances efficiency and productivity
- Reduces risk and complexity
- Lowers cost and increases ROI
- Enables better customer experiences

KEY BUSINESS DRIVERS

1 Modernize Legacy Systems

- Migrate from aging platforms to modern architectures.
- Leverage cloud and new technologies for scalability and innovation.

2 Cloud Adoption and Optimization

- Move data to the cloud for scalability, elasticity and reliability.
- Optimize cloud costs and leverage cloud-native services.

3 Improve Data Quality and Consistency

- Migrate and cleanse data to ensure accuracy, completeness and reliability.
- Establish a trusted single source of truth.

4 Enhance Performance and Scalability

- Improve system performance by migrating to faster, more scalable platforms.
- Support growing data volumes and business needs.

5 Strengthen Security and Compliance

- Migrate to secure platforms to protect sensitive data.
- Meet regulatory requirements and industry standards.

6 Reduce Costs

- Eliminate legacy system maintenance and infrastructure overhead.
- Optimize strorage and operational costs.

7 Support Mergers, Acquisitions and Divestitures

- Consolidate data from multiple sources for integration.
- Enable quick transition and business alignment.

8 Enable Analytics and AI/ML Initiatives

- Migrate data to platforms that support advanced analytics.
- Unlock insights and drive data-driven decision making.

9 Drive Digital Transformation

- Enable modern applications and digital services.
- Empower innovation and improve customer experiences.

10 Ensure Business Continuity and Resilience

- Migrate to reliable, resilient platforms to reduce downtime.
- Improve backup, disaster recovery and data availability.

SUCCESS ENABLERS

- Clear strategy aligned with business goals
- Strong governance and stakeholder alignment
- Comprehensive planning and risk assessment
- Robust data validation and testing
- Proven tools and migration methodologies
- Continuous monitoring and post-migration optimization

COMMON DATA MIGRATION CHALLENGES

Data Migration is Complex – Plan, Prepare and Execute with Confidence

Moving data from one system to another involves more than just copying data. Organizations face multiple challenges that can impact data quality, timelines, costs and business continuity.

Why It Matters

- Ensures data accuracy and reliability
- Helps meet timelines and avoid delays
- Reduces costs and rework
- Minimizes risks and business impact
- Supports successful digital transformation initiatives

1 Data Quality Issues

- Inaccurate, incomplete or duplicate data
- Inconsistent formats and standards
- Poor data profiling before migration

2 Data Mapping Complexity

- Complex source to target mappings
- Different data models and structures
- Business rules not clearly defined

3 Data Volume and Size

- Large volumes of data to be migrated
- Performance and scalability concerns
- Longer migration windows

4 Data Transformation Challenges

- Complex transformations and calculations
- Legacy data cleanup and standardization
- Handling derived and reference data

5 Heterogeneous Systems

- Different source and target technologies
- Compatibility and connectivity issues
- Data type and encoding differences

6 Data Validation and Reconciliation

- Validating migrated data accuracy
- Reconciliation between source and target
- Lack of clear acceptance criteria

7 Security and Compliance Risks

- Protecting sensitive data during migration
- Regulatory compliance requirements
- Access control and audit challenges

8 Downtime and Business Impact

- System downtime during migration
- Impact on ongoing business operations
- Limited maintenance windows

9 Testing and Environment Limitations

- Inadequate testing environments
- Limited test data availability
- Insufficient end-to-end testing

10 Project Planning and Execution Risks

- Unrealistic timelines and estimates
- Scope creep and changing requirements
- Lack of stakeholder alignment

BEST PRACTICES TO OVERCOME CHALLENGES

Plan early and involve all key stakeholders

Profile and cleanse data before migration

Define clear mapping rules and business logic

Validate and reconcile data at every stage of migration

Ensure security, compliance and auditability

Monitor, measure and continuously improve migration processes

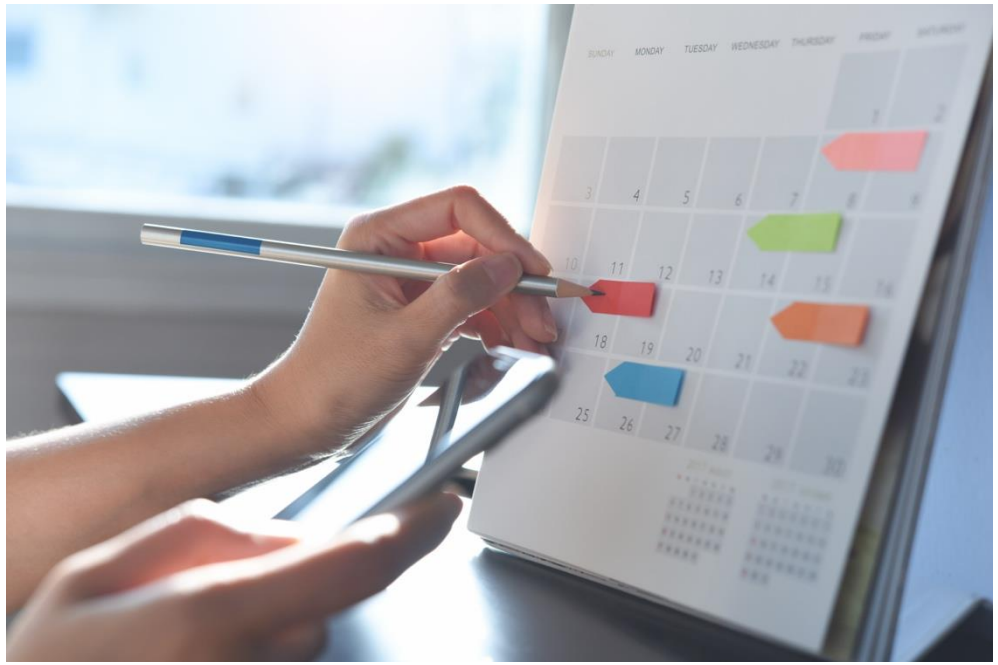
MIGRATION METHODOLOGIES OVERVIEW

Approaches to Move Data and Applications Effectively

Migration methodologies define the strategy and approach for moving data, applications, and workloads from source to target environments. The right methodology ensures data integrity, minimizes downtime, and reduces risk.

Key Benefits

- Reduces risk and downtime
- Ensures data integrity
- Improves efficiency and predictability
- Optimizes cost and resources
- Enables business continuity



COMMON MIGRATION METHODOLOGIES

Methodology	Description	How It Works	Best Use Cases	Advantages	Limitations
Full Migration	Move all data and applications in one single cutover.	• Complete data transfer • System cutover at one point in time • Source system decommissioned	Small to medium databases Low data change rate Short maintenance window	+ Simple to plan and execute + Faster overall project duration + Lower complexity	− Longer downtime − Higher risk for large datasets − Rollback is more difficult
Incremental Migration	Move data in batches or increments, multiple times until complete.	• Initial full load • Subsequent incremental loads • Final sync and cutover	Large databases High data change rate Limited downtime window	+ Reduced downtime + Lower risk + Easier validation	− More complex to implement − Longer overall duration − Requires scheduling
Trickle Migration	Continuously move small transactions in real time.	• Continuous data capture • Real-time replication • Minimal data lag	Mission-critical systems Near-zero downtime High availability required	+ Near-zero downtime + Data always in sync + Business continuity	− High complexity − Requires tooling and resources − Higher cost
Hybrid Migration	Combine multiple methods to fit business and technical needs.	• Mix of full, incremental and trickle approaches • Customized migration plan	Complex environments Multiple systems Varied requirements	+ Flexible and adaptable + Optimized for scenarios + Balanced risk and downtime	− Complex planning − Requires skilled resources − Higher coordination effort

KEY FACTORS FOR SUCCESSFUL MIGRATION

Assessment & Planning
Understand data, applications and dependencies

Data Quality
Cleanse, validate and standardize before migration

Testing & Validation
Thorough testing ensures accuracy and performance

Stakeholder Alignment
Engage stakeholders early and communicate continuously

Monitoring & Support
Monitor, fine-tune and support post migration

HOW TO CHOOSE THE RIGHT METHODOLOGY

- **Data Volume:** How large is the data?
- **Downtime Tolerance:** What is the acceptable downtime?
- **Data Change Rate:** How frequently does data change?
- **System Complexity:** How complex is the environment?
- **Business Impact:** What is the impact on operations?

Evaluate these factors to select the methodology that best fits your business and technical needs.

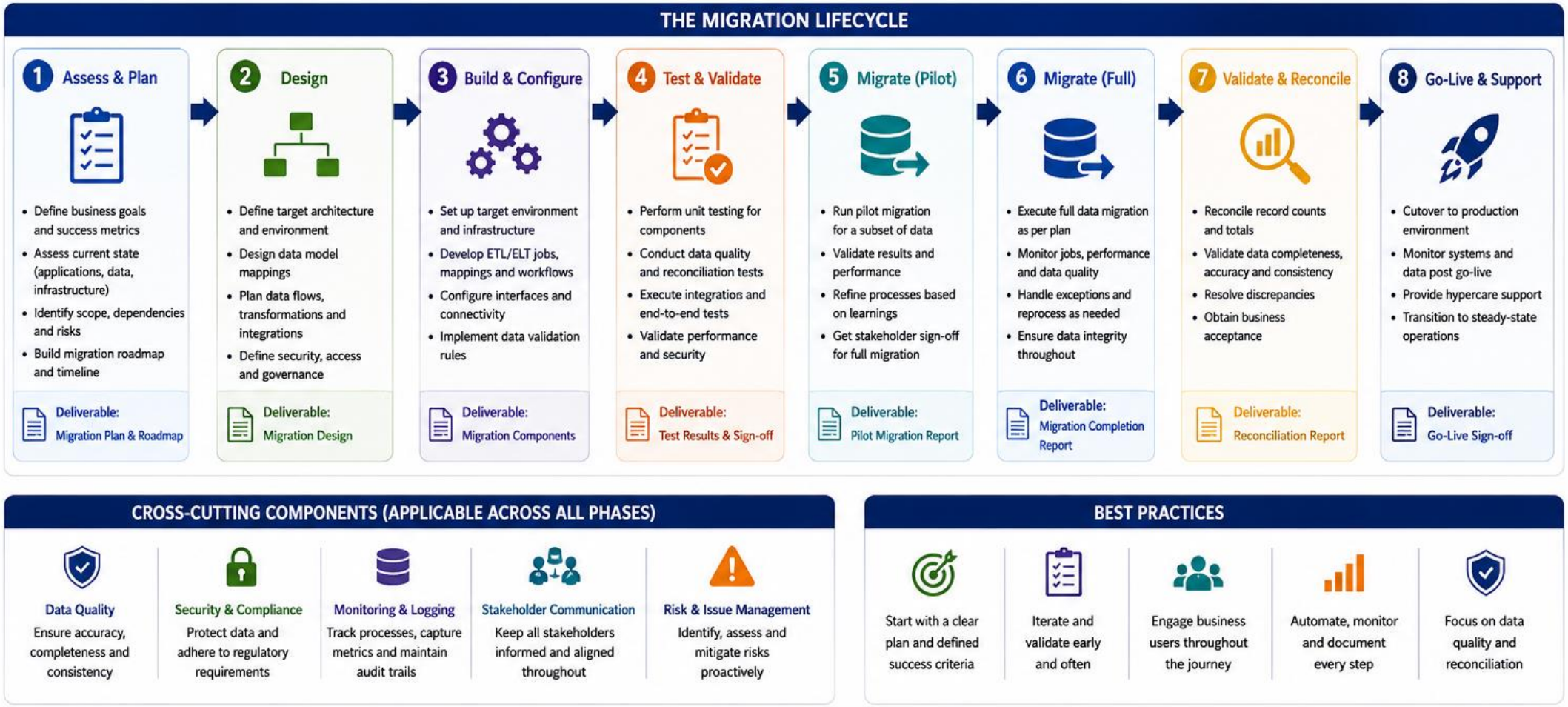
END-TO-END MIGRATION LIFECYCLE

A Structured Approach for Successful Data Migration

A well-defined migration lifecycle ensures data is moved accurately, securely and efficiently from source to target with minimal disruption and maximum business value.

Key Outcomes

- High data accuracy
- Reduced risk and downtime
- Faster time to value
- Optimized cost and resources
- Business user confidence



DATA PROFILING FUNDAMENTALS

Understand Your Data. Improve Quality. Build with Confidence.

Data profiling is the process of analyzing data to understand its structure, content, quality and relationships. It provides deep visibility into data to support better decisions across the data lifecycle.

It answers the key questions:

- What data do we have?
- What does the data look like?
- How good is the data?
- Is the data suitable for its intended use?

HOW DATA PROFILING WORKS

1. Connect
Connect to data sources (DB, files, applications, etc.)

2. Discover
Scan and discover structures, tables and columns

3. Profile
Analyze data content, patterns and quality

4. Analyze
Review metrics, identify issues and risks

5. Act
Take actions to improve data quality and decisions

COMMON PROFILING METRICS

Completeness
% of non-null values in a column.

Uniqueness
% of unique values in a column.

Validity
% of values that conform to defined rules.

Accuracy
% of values that are correct and trusted.

Consistency
Data matches across systems and rules.

Timeliness
How current and up-to-date is the data.

Data Type
Identifies actual data type and variations.

Patterns & Distribution
Understand value distribution and patterns.

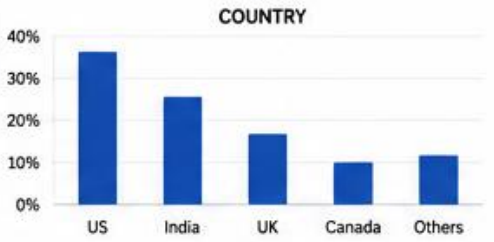
Outliers & Anomalies
Detect unexpected or suspicious values.

EXAMPLE: CUSTOMER PROFILE

Dataset: CUSTOMER

Column Name	Data Type	Completeness	Uniqueness	Invalid Values	Min / Max / Example
CUSTOMER_ID	NUMBER	100%	100%	0	10001 – 99999
FIRST_NAME	VARCHAR(50)	99.2%	92.1%	12	John, Mary, ...
EMAIL	VARCHAR(100)	98.5%	98.3%	45	john@email.com
PHONE	VARCHAR(20)	97.1%	96.8%	87	(555) 123-4567
JOIN_DATE	DATE	100%	–	0	2018-01-01 / Today
COUNTRY	VARCHAR(50)	99.8%	85.4%	5	US, UK, India, ...

Value Distribution (Example)



Data Quality Score

Insights

- EMAIL has 45 invalid values – requires cleansing.
- PHONE has 87 invalid values and inconsistent formats.
- COUNTRY has good coverage with few missing values.

Recommended Actions

- ☑ Standardize phone format
- ☑ Validate email using regex
- ☑ Enrich missing country values

DATA CLEANSING TECHNIQUES

Clean Data. Trusted Insights. Better Decisions.

Data cleansing is the process of detecting and correcting inaccurate, incomplete, inconsistent and duplicate data to improve data quality and reliability.

Why It Matters

- Improves data quality and accuracy
- Enhances analysis and decision-making
- Reduces costs and rework
- Ensures compliance and reduces risk
- Builds trust in data and reports

COMMON DATA QUALITY ISSUES

Missing Values
Important data is blank or not available.

Inconsistent Values
Same data stored in different formats (e.g., "NY", "New York").

Duplicate Records
Same record appears multiple times.

Invalid Data
Data that violates rules or business constraints (e.g., age = -5).

Outdated Data
Data that is no longer current or relevant.

DATA CLEANSING PROCESS

1. **Profile** — Profile data to identify quality issues and patterns.
2. **Identify** — Detect and quantify issues based on rules and thresholds.
3. **Cleanse** — Apply cleansing techniques to correct or standardize data.
4. **Validate** — Validate results to ensure data quality improvement.
5. **Monitor** — Continuously monitor and maintain data quality over time.

KEY DATA CLEANSING TECHNIQUES

Standardization
AB → Ab
Convert data to a consistent format, case, and structure.

Deduplication
Identify and remove duplicate records.

Missing Value Handling
Impute, default, or remove missing or null values.

Validation
Validate data against rules, ranges, formats and business constraints.

Normalization
Transform data into a standard model or range.

Data Type Correction
123 → 123
Correct data types and format mismatches.

Outlier Handling
Detect and handle outliers based on business rules.

Enrichment
Enhance data with additional accurate and relevant information.

EXAMPLE: CUSTOMER DATA

Before Cleansing

Cust_ID	Name	Email	City	Phone	Age
1	john doe	john.doe@gmail.com	ny	555-1234	29
2	Jane Smith	jane.smith@Gmail.com	New York	5551234	-5
3	JOHN DOE	john.doe@gmail.com	NY	555-1234	29
4	Mike Brown	mike.brown@yahoo.com	Los angeles	555-5678	35
5	Jane Smith		new york	555-1234	35

After Cleansing

Cust_ID	Name	Email	City	Phone	Age
1	John Doe	john.doe@gmail.com	New York	555-1234	29
2	Jane Smith	jane.smith@gmail.com	New York	555-1234	35
3	Mike Brown	mike.brown@yahoo.com	Los Angeles	555-5678	35

KEY BENEFITS

- Reliable, accurate and consistent data for trusted reporting.
- Better customer, operational and financial insights.
- Improved efficiency and reduced manual rework.
- Stronger compliance and lower risk of bad decisions.

BEST PRACTICES

- Define clear data quality rules and business standards.
- Profile data regularly to detect issues early.
- Use automated tools wherever possible to scale.
- Involve business stakeholders in defining and validating rules.
- Monitor data quality continuously and track improvements.
- Document rules, exceptions and cleansing logic for maintainability.

SOURCE-TO-TARGET MAPPING

Define How Data Flows from Source to Target

Source-to-Target mapping defines how data elements from the source system are transformed and loaded into the target system. It ensures data accuracy, consistency, and integrity during migration.

Why It Matters

- Ensures accurate data migration
- Maintains data integrity
- Handles data transformations
- Enables better reporting & analysis

KEY CONCEPTS

Source System
The system where data currently resides.

Target System
The system where data will be migrated.

Transformation
Rules and expressions applied to convert source data to target format.

Mapping Rule
Defines the relationship between source and target fields.

Unmapped / Default
Fields without a source or using a default value.

MAPPING EXAMPLE

SOURCE SYSTEM (Oracle) → MAPPING RULE / TRANSFORMATION → TARGET SYSTEM (Salesforce)

Source Field	Source Data Type	Mapping Rule / Transformation	Target Field	Target Data Type
CUST_ID	NUMBER(10)	Direct Map	Customer_Id__c	Text(10)
CUST_NAME	VARCHAR2(100)	Direct Map	Name	Text(80)
FIRST_NAME	VARCHAR2(50)	Direct Map	FirstName	Text(40)
LAST_NAME	VARCHAR2(50)	Direct Map	LastName	Text(40)
EMAIL_ID	VARCHAR2(100)	LOWER(TRIM(EMAIL_ID))	Email	Email
PHONE_NO	VARCHAR2(20)	REGEXP_REPLACE(PHONE_NO, '[^0-9]', '')	Phone	Phone
GENDER	VARCHAR2(1)	DECODE(GENDER, 'M', 'Male', 'F', 'Female', 'U', 'Unknown')	Gender__c	Picklist
DOB	DATE	TO_CHAR(DOB, 'YYYY-MM-DD')	Date_of_Birth__c	Date
ADDRESS	VARCHAR2(255)	SUBSTR(ADDRESS, 1, 255)	MailingStreet	Text Area(255)
CREATED_DATE	DATE	Direct Map	CreatedDate	Date/Time
LOYALTY_POINTS	NUMBER(10)	NVL(LOYALTY_POINTS, 0)	LoyaltyPoints__c	Number(10,0)
COMMENTS	VARCHAR2(4000)	— Not Mapped —	—	—

BEST PRACTICES

- Involve business and technical stakeholders in mapping.
- Validate and test mappings thoroughly.
- Document all mapping rules and assumptions.
- Handle exceptions and defaults explicitly.

LEGEND

- Direct Mapping
- Transformation Applied
- Default / Derived Value
- Not Mapped

MAPPING PROCESS

1. Analyze Source and Target Structures
2. Identify and Define Mapping Rules
3. Build and Configure Transformations
4. Validate and Test Mappings
5. Load and Reconcile Data
6. Monitor and Optimize

DATA VALIDATION STRATEGIES

Ensure Data Quality, Accuracy and Consistency

Data validation ensures that the data being processed is accurate, complete, consistent and complies with business rules before it is loaded or used in downstream processes.

Why It Matters

- Improves data quality
- Prevents bad data from entering the system
- Ensures reliable reporting and decision making
- Reduces cost of data errors and rework
- Builds trust in data and systems

COMMON VALIDATION TYPES

Format Validation
Ensures data matches the expected format, pattern or data type.

Range Validation
Checks that data falls within acceptable minimum and maximum values.

Referential Validation
Verifies that data exists in a related reference or lookup source.

Completeness Validation
Ensures required fields are not null or blank.

Business Rule Validation
Applies business rules and cross-field logic to ensure data is valid.

Uniqueness Validation
Checks that data does not create duplicates based on defined keys.

DATA VALIDATION STRATEGY

1. Define Rules
Identify validation rules based on business and data quality requirements.

2. Validate Early
Validate data as early as possible in the data pipeline (Extract/Stage).

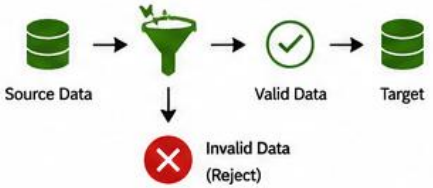
3. Handle Failures
Identify invalid data and route to reject or quarantine locations.

4. Correct & Cleanse
Fix data issues or apply cleansing rules where appropriate.

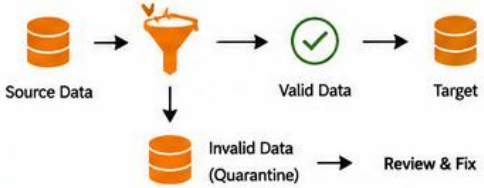
5. Monitor & Improve
Continuously monitor validation results and refine rules for improvement.

VALIDATION APPROACH

Reject Strategy
Invalid data is rejected and not loaded.



Quarantine Strategy
Invalid data is stored separately for review.



BEST PRACTICES

- Define clear and measurable validation rules.
- Validate as early as possible in the process.
- Use reusable validation components and mapplets.
- Maintain a data quality dashboard and alerts.
- Log and track all validation failures for audit and analysis.
- Periodically review and update validation rules.

EXAMPLE VALIDATIONS

Validation Type	Example Rule	Description	Example Check	Action on Failure
Format Validation	Email Format	Email must be in valid email format	`REGEXP_LIKE(EMAIL, '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,$')`	Reject / Quarantine
Range Validation	Age Range	Age must be between 0 and 120	`AGE >= 0 AND AGE <= 120`	Reject / Quarantine
Referential Validation	Country Code	Country code must exist in reference table	`EXISTS (SELECT 1 PROM REF_COUNTRY WHERE CODE = SRC.COUNTRY_CODE)`	Reject / Quarantine
Completeness Validation	Mandatory Fields	Customer ID should not be null	`CUSTOMER_ID IS NOT NULL`	Reject / Quarantine
Business Rule Validation	Order Date Rule	Order date cannot be in the future	`ORDER_DATE <= CURRENT_DATE`	Reject / Quarantine
Uniqueness Validation	Unique Customer	Customer ID must be unique	`COUNT(*) OVER (PARTITION BY CUSTOMER_ID) = 1`	Reject / Quarantine

USER ACCEPTANCE TESTING

Validating the Solution. Delivering Business Value.

User Acceptance Testing (UAT) is the final testing phase where real users validate the solution against business requirements to ensure it is ready for production and meets user needs.

Why It Matters

- Ensures solution meets business requirements
- Validates from end-user perspective
- Reduces risk of defects and rework
- Increases user confidence and adoption
- Ensures readiness for production and success

WHAT IS UAT?

UAT is performed by end users or business representatives to evaluate whether the solution supports real business processes and is ready for live use.

UAT OBJECTIVES

Validate business requirements
Confirm the solution meets defined business needs.

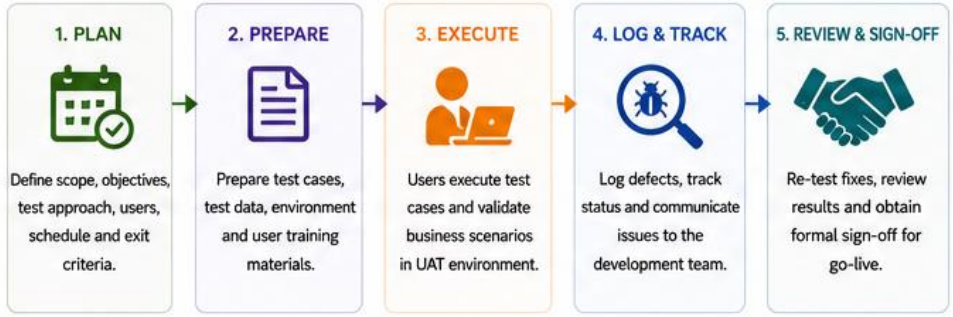
Evaluate usability
Ensure the system is easy to use and supports daily operations.

Verify business workflows
Validate end-to-end processes in a realistic environment.

Confirm data accuracy
Ensure data is correct, complete and integrated as expected.

Provide sign-off for go-live
Obtain formal approval to move the solution to production.

UAT PROCESS



TYPES OF UAT

UAT CHECKLIST

- [x] Business requirements are fully understood
- [x] UAT test cases are prepared and reviewed
- [x] Test data is accurate and complete
- [x] UAT environment is stable and production-like
- [x] End users are trained and ready
- [x] Defects are logged and resolved
- [x] All critical scenarios are successfully validated
- [x] Business sign-off is obtained

ROLES & RESPONSIBILITIES

ROLE	RESPONSIBILITY
End Users / Business Users	Execute test cases, validate and provide feedback
UAT Coordinator	Plan UAT activities and manage execution
QA / Test Team	Support UAT, manage defects and retesting
Developers	Fix defects and provide support
Business Owner	Review results and provide final sign-off

BEST PRACTICES

- Involve end users early in planning and test scenario definition.
- Keep test cases clear, simple and business focused.
- Use realistic test data that covers key business scenarios.
- Ensure clear communication and quick resolution of defects.
- Track progress daily and monitor exit criteria closely.
- Provide adequate training and support to end users.

ORACLE TO MONGODB MIGRATION CASE STUDY

Case Study: Modernizing for Agility, Scalability and Performance

A leading e-commerce company migrated its product catalog and customer interaction data from Oracle Database to MongoDB to support agility, scale and faster time-to-market.

BUSINESS CONTEXT

- The company faced performance bottlenecks and high licensing costs with Oracle.
- Business needed flexible schema, horizontal scalability and better support for semi-structured data (product details, reviews, user preferences).

OBJECTIVES

- Migrate product catalog and customer interaction data from Oracle to MongoDB.
- Improve performance and scalability.
- Reduce total cost of ownership.
- Enable agile development and rapid innovation.

SCOPE

- ~2 TB data migrated from Oracle 19c.
- 120+ tables, 350+ stored procedures.
- Data domains: Product Catalog, Inventory, Customer Reviews, User Activity, Preferences.
- Batch migration with minimal application downtime.

TECHNOLOGY STACK

- Source: Oracle Database 19c
- Target: MongoDB Atlas (Sharded Cluster)
- ETL/Migration Tool: Informatica PowerCenter
- Validation: Custom Python & MongoDB Tools
- Reporting: Tableau

MIGRATION APPROACH

1. **Assess & Plan** — Assess application, data, dependencies and design migration roadmap.
2. **Data Profiling & Assessment** — Profile data, identify quality issues and determine migration strategy.
3. **Schema Mapping** — Map relational schema to MongoDB document model.
4. **Data Migration** — Extract, transform and load data in batches with incremental loads.
5. **Validation & Reconciliation** — Validate data completeness, accuracy and business rules.
6. **Cutover & Go-Live** — Perform final sync, switch application and monitor stabilization.

KEY MIGRATION ACTIVITIES

- Analyzed 120+ tables and relationships.
- Designed optimized MongoDB document models.
- Handled complex transformations and data type conversions.
- Implemented data cleansing and deduplication.
- Migrated historical data in batches.
- Performed extensive validation and reconciliation.
- Cutover executed with minimal downtime (< 30 minutes).

ORACLE TO MONGODB MAPPING EXAMPLE

Oracle (Relational)

Example Table: PRODUCT

PRODUCT_ID (PK)
PRODUCT_NAME
CATEGORY_ID (FK)
PRICE
DESCRIPTION
ATTRIBUTES (multiple rows)
INVENTORY_QTY

MongoDB (Document)

Collection: products

```
{
 "_id": ObjectId(".."),
 "productName": "Wireless Headphone",
 "category": {
    "id": 10,
    "name": "Electronics"
 },
 "price": 129.99,
 "description": "High quality wireless
    headphone with noise cancellation",
 "attributes": [
    { "name": "Color", "value": "Black" },
    { "name": "Battery", "value": "20H" }
 ],
 "inventoryQty": 250
}
```

MIGRATION RISK MANAGEMENT

Identify, Assess and Mitigate Risks for Successful Migration

Effective risk management ensures data integrity, minimal downtime, security, and business continuity throughout the migration lifecycle.

- Identify risks early in the migration lifecycle.
- Assess impact and likelihood.
- Implement proactive mitigation strategies.
- Continuously monitor and adapt.
- A well-managed risk plan ensures a smooth and successful migration.

Risk Category	Description	Impact	Mitigation Strategies
 **Data Loss or Corruption**	• Data may be lost or corrupted during extraction, transfer, or loading.	**High** Loss of critical business data	• Data validation at every stage • Use checksums and data profiling • Maintain backups and point-in-time recovery
 **Downtime and Service Disruption**	• Migration activities may cause application downtime or degraded performance.	**High** Business interruption and SLA breaches	• Plan cutover during low-traffic hours • Use replication / CDC for minimal downtime • Perform thorough testing before go-live
 **Security and Compliance Risks**	• Sensitive data exposure or non-compliance with regulations.	**High** Legal, financial and reputational impact	• Encrypt data in transit and at rest • Enforce IAM roles and least privilege • Ensure compliance with GDPR, PIPEDA, HIPAA, etc.
 **Performance Risks**	• Poor performance in target environment due to improper sizing or configuration.	**Medium** Slow applications and poor user experience	• Right-size infrastructure • Optimize indexes and queries • Conduct performance testing
 **Compatibility and Integration Risks**	• Incompatibility with applications, drivers, or third-party integrations.	**Medium** Functionality issues and integration failures	• Validate compatibility early • Test integrations in staging environment • Use compatible drivers and APIs
 **People and Process Risks**	• Lack of skills, unclear roles or poor communication.	**Medium** Project delays and errors	• Define clear roles and responsibilities • Provide training and documentation • Maintain regular communication

CUTOVER PLANNING AND EXECUTION

Seamless transition from source to target with minimal disruption and business impact.

Always perform a rehearsal (dress rehearsal) in lower environments to identify risks and validate the cutover plan before the actual cutover.

A successful cutover is the result of careful planning, clear communication, disciplined execution, thorough validation, and strong post-cutover support.

CUTOVER APPROACH

Big Bang

Single cutover at a specific point in time. Simple but higher risk.

Trickle (Phased)

Data migrated in phases/categories. Lower risk, more time.

Dual Write

Write to both source and target during transition. More control, more complexity.

Change Data Capture (CDC)

Continuous sync of changes until cutover. Ideal for minimal downtime.

CUTOVER OBJECTIVES

- Minimize downtime and business disruption
- Ensure complete and accurate data migration
- Maintain data integrity and consistency
- Enable quick rollback if needed
- Validate business readiness post cutover

CUTOVER PLANNING CHECKLIST

1. Define Cutover Window
Agree on cutover date, time, and downtime window with all stakeholders.

2. Pre-Cutover Validation
Validate data integrity, reconciliation reports and performance benchmarks.

3. Communication Plan
Inform all stakeholders, users and support teams about cutover activities.

4. Backup & Rollback Plan
Take final backups and verify ability to rollback if required.

5. Cutover Readiness Review
Confirm all tasks completed and environment is ready for cutover.

6. Go / No-Go Decision
Final approval from stakeholders to proceed with cutover.

CUTOVER EXECUTION STEPS

1 Pre-Cutover Activities
Stop non-essential updates, run final incremental loads, validate data.

2 Quiesce Source Systems
Put source systems in read-only mode or stop transactions.

3 Final Data Sync
Run final full or incremental data load and apply changes.

4 Switch Over
Redirect application connections to target environment.

5 Post-Cutover Validation
Validate data, test critical transactions and performance.

6 Hypercare & Monitoring
Monitor systems closely and resolve any issues.

POST CUTOVER ACTIVITIES

Data Validation
Reconcile record counts, key data and business reports.

Performance Monitoring
Monitor system performance and user experience.

User Acceptance
Confirm business users can operate in the new environment.

Decommission Source
Archive or decommission source systems when stable.

SUCCESS FACTORS

- Detailed planning and rehearsals
- Clear communication
- Automated and tested processes
- Strong monitoring and support
- Well defined rollback plan

POST-MIGRATION VALIDATION

Best Practices

- Validate in phases: small batches before full migration.
- Use automated validation scripts.
- Generate validation reports and share with stakeholders.
- Investigate and resolve all issues before go-live.
- Re-run validation after any re-migration or retry.

MIGRATION BEST PRACTICES

Proven practices for successful, secure, and reliable data migrations to MongoDB.

Key Areas of Focus

- Ensure data is complete, accurate, and consistent.
- Build repeatable processes and plan for failure.
- Optimize for speed and scalability.
- Protect data and ensure compliance.
- Align teams, processes and communication.

1 PLAN AND ASSESS THOROUGHLY

- Define clear goals, scope, and success metrics.
- Assess data, applications, and dependencies.
- Identify risks early and plan mitigations.

2 UNDERSTAND YOUR DATA

- Profile data for volume, quality, and patterns.
- Identify relationships and data dependencies.
- Classify data (critical, historical, archival).

3 DESIGN THE RIGHT TARGET MODEL

- Choose the right data model (embedded vs. referenced).
- Align with query patterns and use cases.
- Avoid over-normalization.

4 CHOOSE THE RIGHT TOOLS AND APPROACH

- Use the right migration tools and frameworks.
- Prefer automation and repeatable processes.
- Avidate tool compatibility and performance.

5 PREPARE AND CLEANSE YOUR DATA

- Standardize formats and naming conventions.
- Handle nulls, duplicates, and inconsistent data.
- Enrich or transform data as needed.

6 SECURITY AND COMPLIANCE FIRST

- Protect data in transit and at rest.
- Follow least-privilege access principles.
- Ensure compliance with policies and regulations.

7 TEST EARLY AND OFTEN

- Perform pilot migrations with real data.
- Validate mappings, transformations, and business rules.
- Fix issues before full-scale migration.

8 PERFORMANCE AND SCALABILITY

- Benchmark and optimize for scale.
- Use batching and parallelization wisely.
- Monitor resource usage and tune.

9 VALIDATE COMPLETELY

- Verify record counts and checksums.
- Validate data integrity and referential consistency.
- Reconcile business totals and KPIs.

10 MONITOR AND OBSERVE

- Monitor migration jobs in real time.
- Track logs, metrics, and error rates.
- Set alerts and proactively resolve issues.

11 PLAN FOR ROLLBACK AND RECOVERY

- Have a rollback plan and tested backups.
- Keep source data intact until validation is complete.
- Document recovery procedures.

12 ENABLE TEAMS AND COMMUNICATE

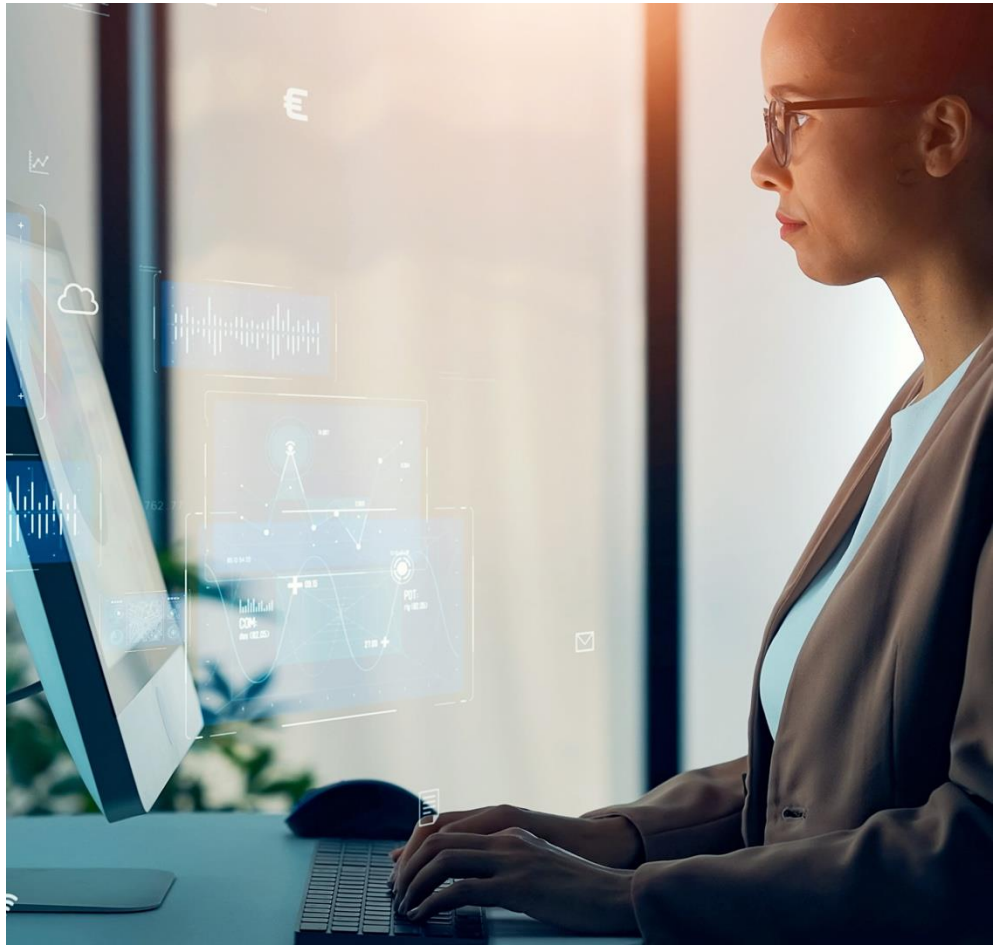
- Engage stakeholders and keep them informed.
- Provide training and clear documentation.
- Foster collaboration across teams.

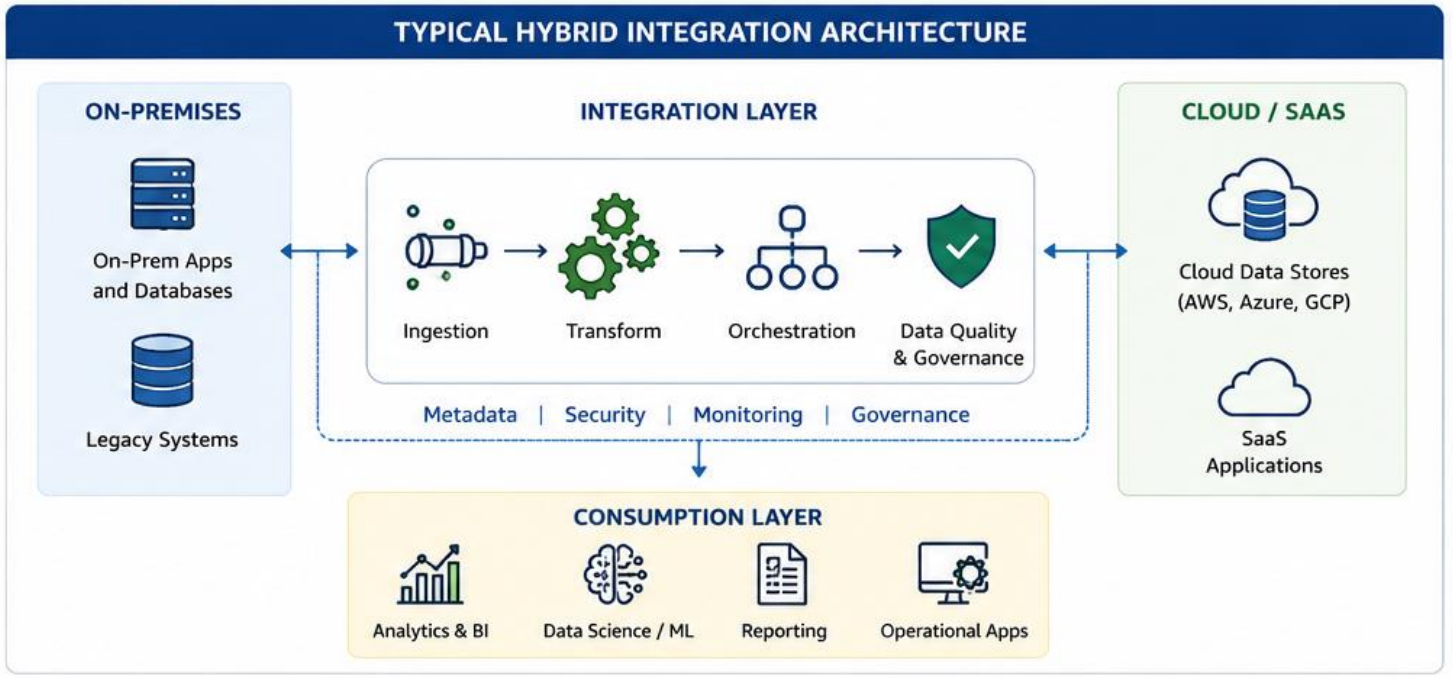
MODULE 4 — HYBRID DATA INTEGRATION

INTRODUCTION TO HYBRID DATA INTEGRATION

Hybrid Data Integration connects and orchestrates data across different environments—on-premises, cloud, SaaS, and databases—enabling seamless data flow, consistency, and real-time or batch processing.

- Unified View of Data – Break down silos and get a 360° view.
- Flexibility – Integrate diverse sources and technologies.
- Scalability – Handle growing data volumes and complexity.
- Real-Time Insights – Enable faster, data-driven decisions.
- Business Agility – Quickly adapt to changing business needs.





KEY COMPONENTS

- **Connectors & Adapters** – Connect to various data sources and destinations
- **ETL/ELT Processes** – Extract, transform, and load data efficiently
- **Orchestration** – Schedule, automate, and monitor data workflows
- **Data Quality** – Validate, cleanse, and enrich data
- **Security** – Protect data in transit and at rest
- **Governance** – Ensure compliance, lineage, and policy management

KEY CHALLENGES

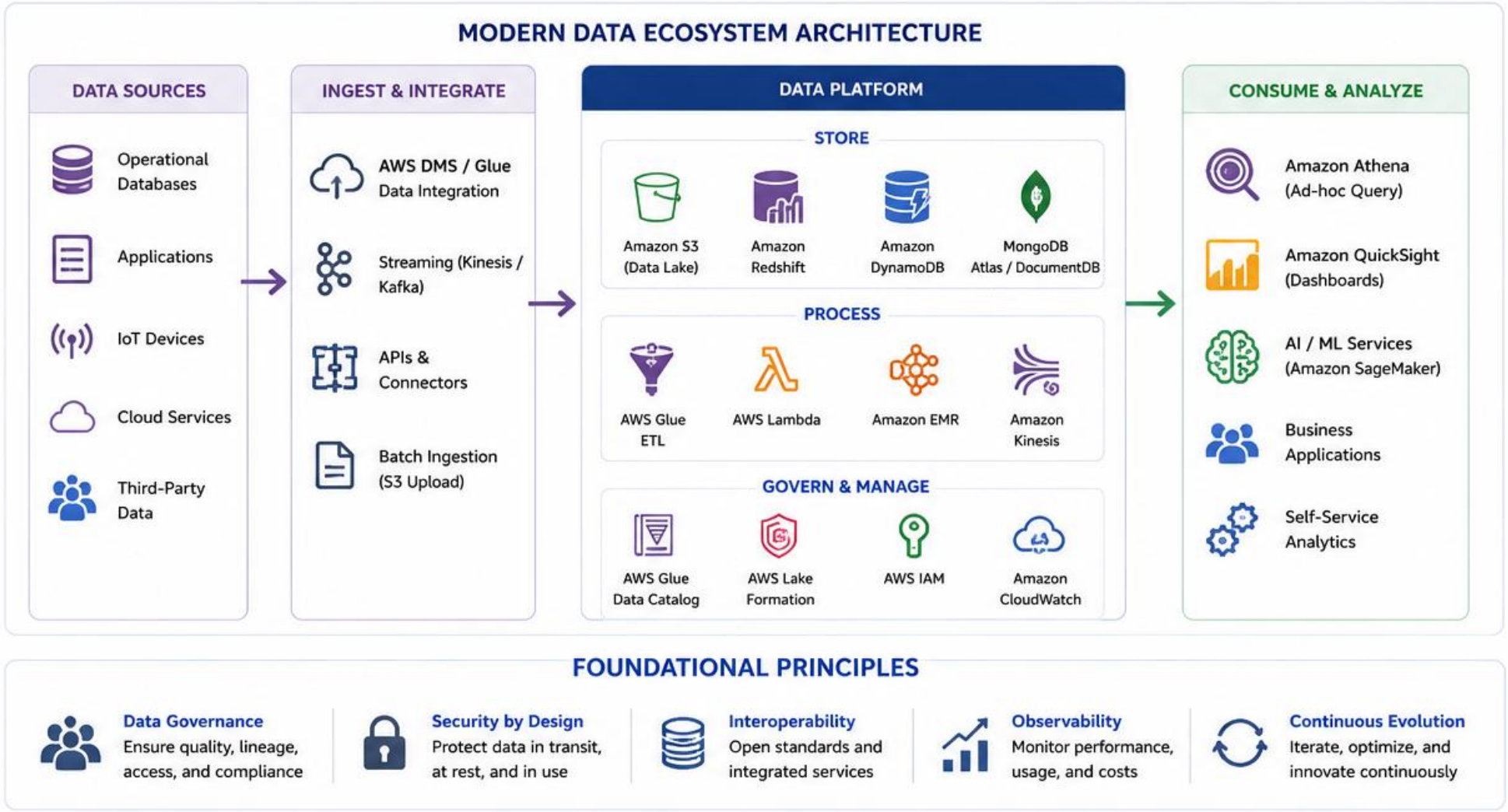
- Data consistency across environments
- Network latency and connectivity
- Security and compliance management
- Heterogeneous data formats and schemas
- Complexity in orchestration and monitoring

MODERN ENTERPRISE DATA ECOSYSTEMS

Integrated, scalable, and intelligent ecosystems that turn data into business value.

KEY CHARACTERISTICS

- Connected: Seamless integration across platforms
- Scalable: Elastic infrastructure for growth
- Secure: Governed, compliant, and trusted
- Intelligent: AI/ML and analytics drive insights
- Collaborative: Enabled for self-service and collaboration



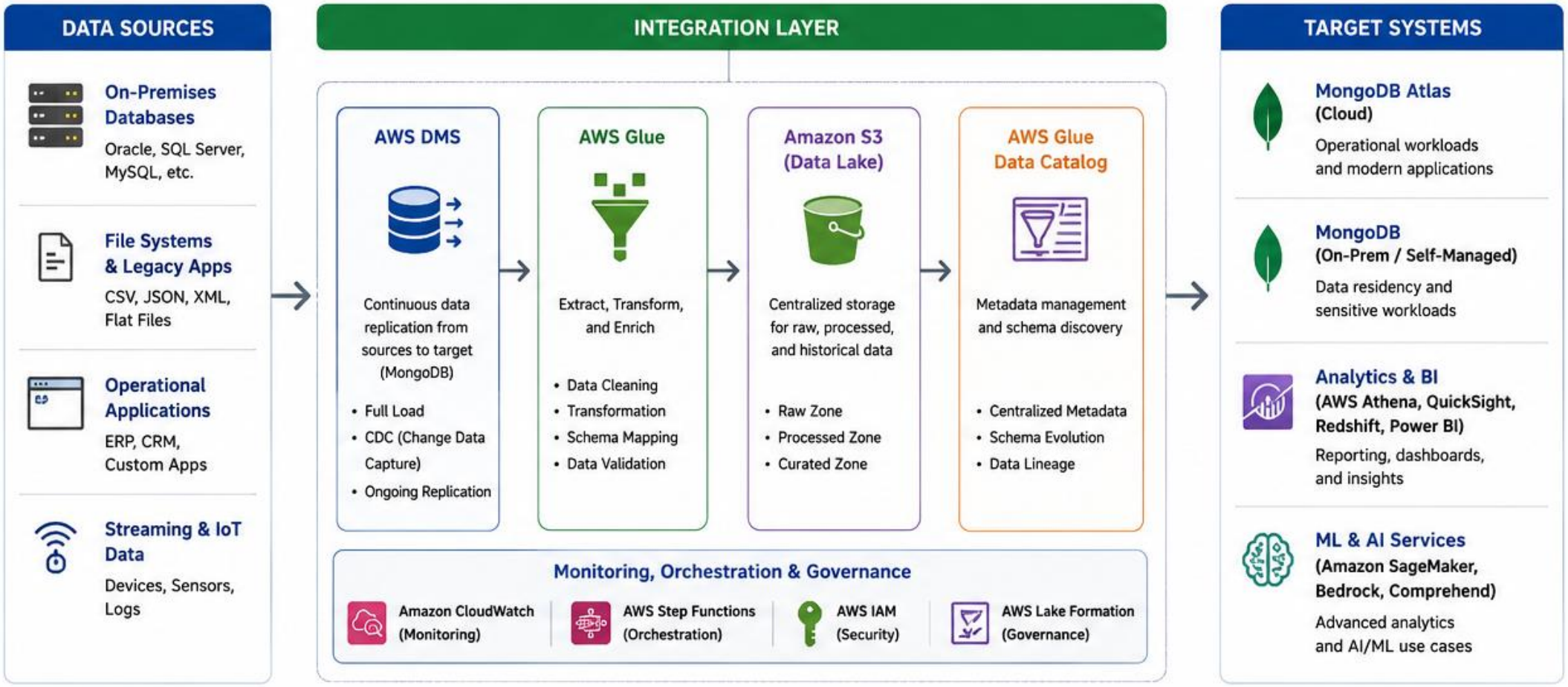
HYBRID INTEGRATION ARCHITECTURE

HYBRID INTEGRATION ARCHITECTURE

Seamlessly integrate data across on-premises systems, MongoDB, and cloud data platforms.

Key Integration Patters

- Batch Migration
- CDC Replication
- Event-Driven Integration
- ETL/ELT Pipelines
- Data Virtualization



ON-PREMISES VS CLOUD SYSTEMS

Key differences to consider for data migration and modernization decisions

Key Takeways

- Choose based on business goals, budget, and technical needs.
- Many organizations adopt a hybrid approach.
- Cloud offers agility and flexibility.
- Security and compliance are essential in both models.
- Plan migration strategically for maximum value.

ON-PREMISES SYSTEMS	ASPECT	CLOUD SYSTEMS
Infrastructure Ownership You own and manage physical hardware and infrastructure.	**Ownership**	**Provider-managed infrastructure** Cloud provider owns and manages the infrastructure.
Cost Model High upfront capital expenditure (CapEx) plus ongoing operational expenses.	**Cost**	**Pay-as-you-go (OpEx)** Pay only for what you use; lower upfront costs.
Scalability Limited scalability; requires purchasing and installing additional hardware.	**Scalability**	**Highly scalable** Scale up or down quickly and automatically.
Maintenance You are responsible for hardware, software, updates, and troubleshooting.	**Maintenance**	**Provider handles maintenance** Automatic updates, patches, and managed services.
Security You are fully responsible for security, compliance, and data protection.	**Security**	**Shared responsibility model** Provider secures the cloud; you secure your data and access.
Control & Customization Full control over hardware, software, and configurations.	**Control**	**Less control over underlying infrastructure** More control over services and configurations.
Deployment Speed Slow provisioning; weeks or months to deploy new systems.	**Speed**	**Fast provisioning** Deploy resources in minutes.
Data Location Data resides in your data center.	**Data Location**	**Global data centers** Data stored in provider regions around the world.
Disaster Recovery You build and manage DR solutions; higher cost and complexity.	**Disaster Recovery**	**Built-in DR options** Multi-AZ/region backups and high availability.
Best Suited For Stable workloads, strict compliance needs, legacy systems.	**Best Suited For**	**Dynamic workloads, innovation, flexibility, global scale.**

INTEGRATION PATTERNS OVERVIEW

Common patterns used to integrate MongoDB with enterprise and cloud data ecosystems.

Key Considerations

- Data Volume & Velocity
- Data Consistency Requirements
- Security & Compliance
- Cost Implications
- Tooling & Ecosystem

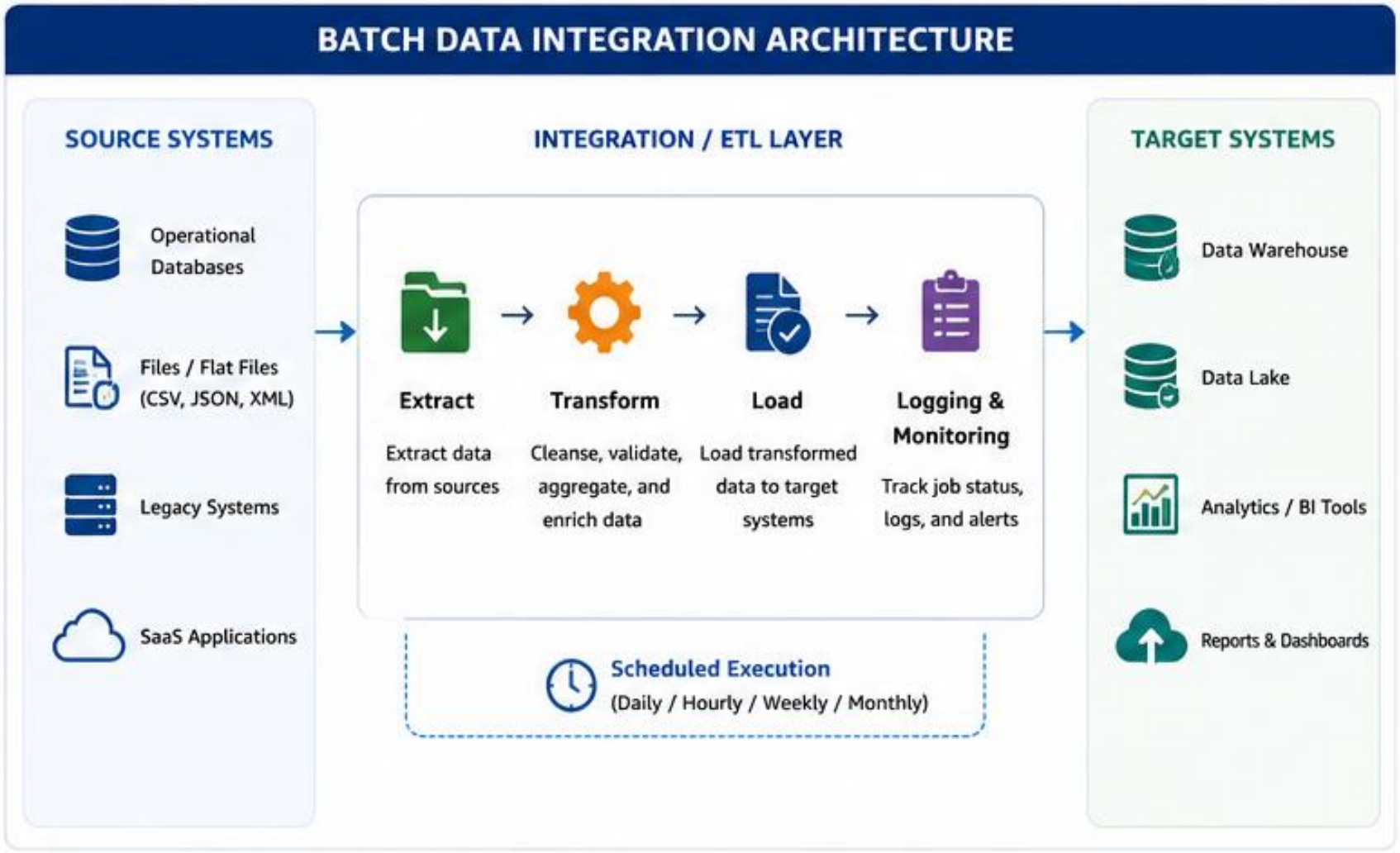
PATTERN	DESCRIPTION	TYPICAL USE CASES	PROS	CONSIDERATIONS
1  **Direct Application Integration**	Applications connect directly to MongoDB using native drivers.	• Operational applications • Real-time transactions • Mobile and web apps	✓ Simple and fast ✓ Full feature access ✓ Real-time data access	⚠ Tight coupling ⚠ App changes may be required
2  **ETL / ELT Integration**	Data is extracted, transformed, and loaded into or from MongoDB.	• Data warehousing • Reporting & analytics • Data lake integration	✓ Handles large volumes ✓ Supports complex transformations ✓ Batch oriented	⚠ Higher latency ⚠ Operational overhead ⚠ Batch windows
3  **Change Data Capture (CDC)**	Capture and stream real-time changes from MongoDB (or sources) to target systems.	• Real-time analytics • Sync with data lake • Search index updates • Event-driven workflows	✓ Near real-time sync ✓ Minimal source impact ✓ Scalable	⚠ More complex setup ⚠ Requires monitoring ⚠ Ordering & lag
4  **Event-Driven Integration**	MongoDB changes emit events to a message bus (e.g., Kafka, SNS).	• Microservices integration • Real-time notifications • Decoupled architectures	✓ Highly decoupled ✓ Scalable & resilient ✓ Real-time	⚠ Event ordering ⚠ Event duplication ⚠ Requires governance
5  **API-Based Integration**	Expose or consume data via REST/GraphQL APIs that interact with MongoDB.	• Partner integrations • SaaS platforms • Cross-system access	✓ Language agnostic ✓ Secure & controlled ✓ Easy to evolve	⚠ API latency ⚠ Throttling limits ⚠ Additional layer
6  **Data Federation / Virtualization**	Query data in MongoDB alongside other sources without moving data.	• Unified analytics views • Multi-source reporting • Data mesh architectures	✓ No data duplication ✓ Real-time unified views ✓ Flexible	⚠ Performance depends on sources ⚠ Complex queries

BATCH DATA INTEGRATION

Integrate data in batches at scheduled intervals for reliable, efficient, and cost-effective processing.

Common Use Cases

- Daily / Periodic Reporting
- Data Warehouse Loading
- History & Archive Processing
- Data Consolidation Across Systems
- Regulatory & Compliance Reporting

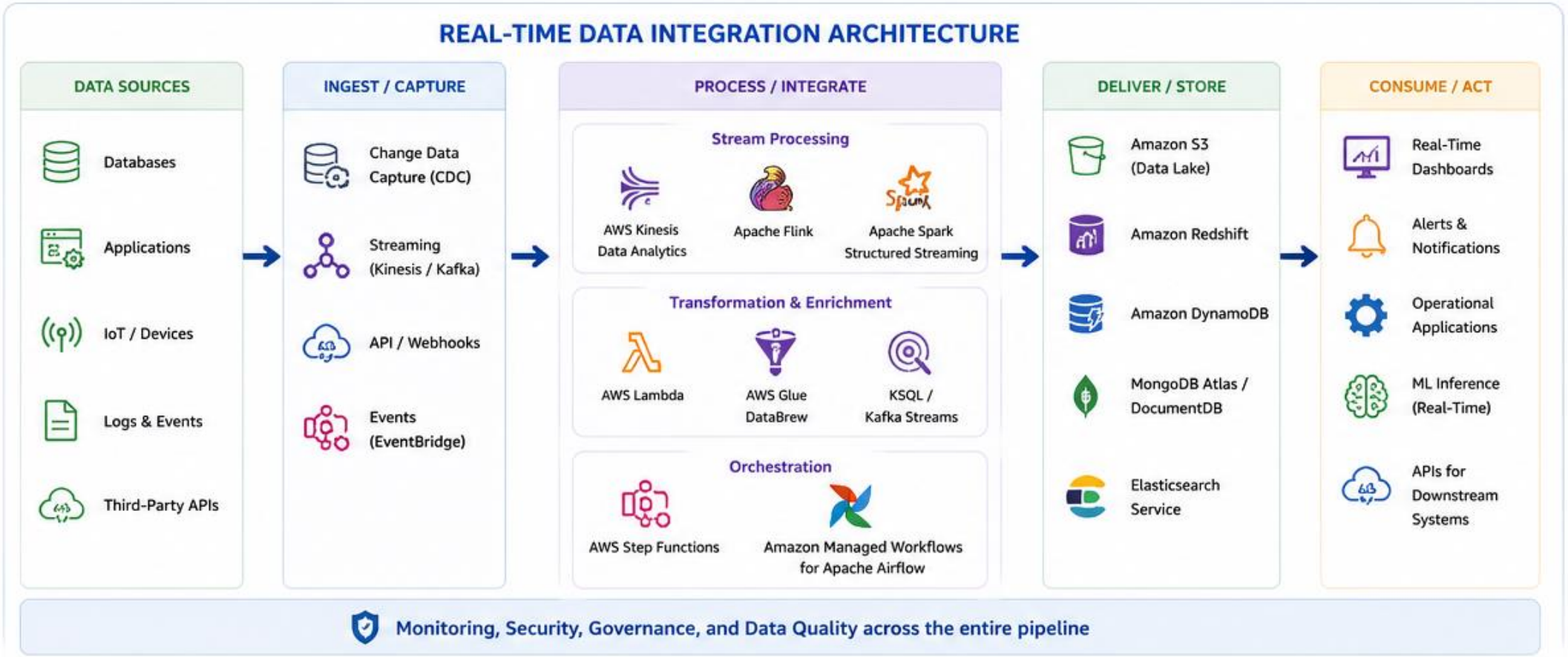


REAL-TIME DATA INTEGRATION

Deliver the right data to the right place at the right time for real-time decisions.

Common Use Cases

- Real-Time Customer 360
- Fraud Detection
- IoT Analytics
- Real-Time Operations
- Event-Driven Automation



API INTEGRATION

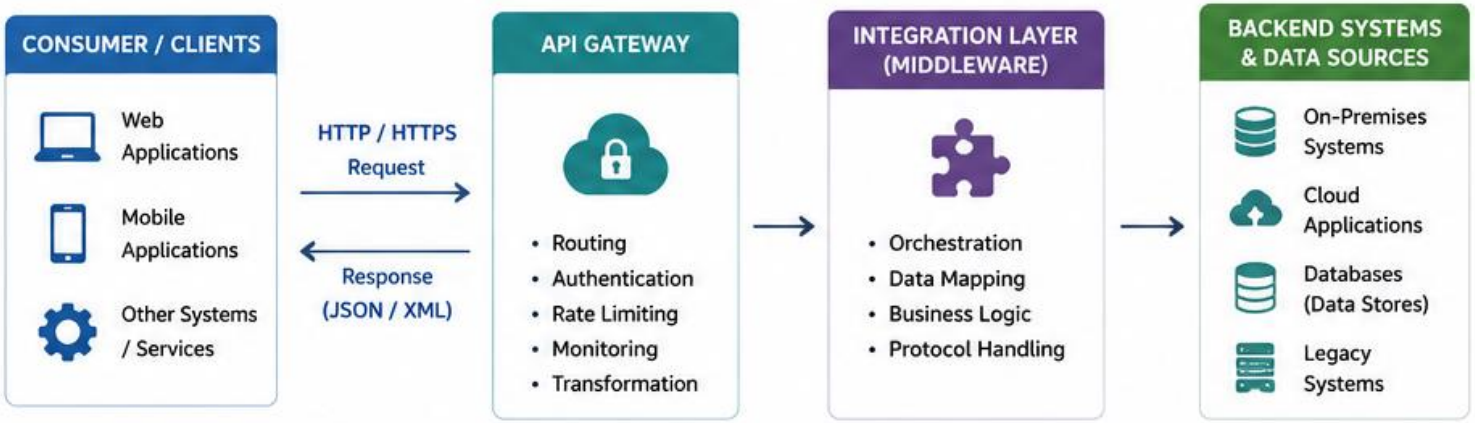
Connect systems, applications, and data in real time using standardized APIs.

APIs enable secure, scalable, and reliable communication between on-premises and cloud systems, applications, and services.

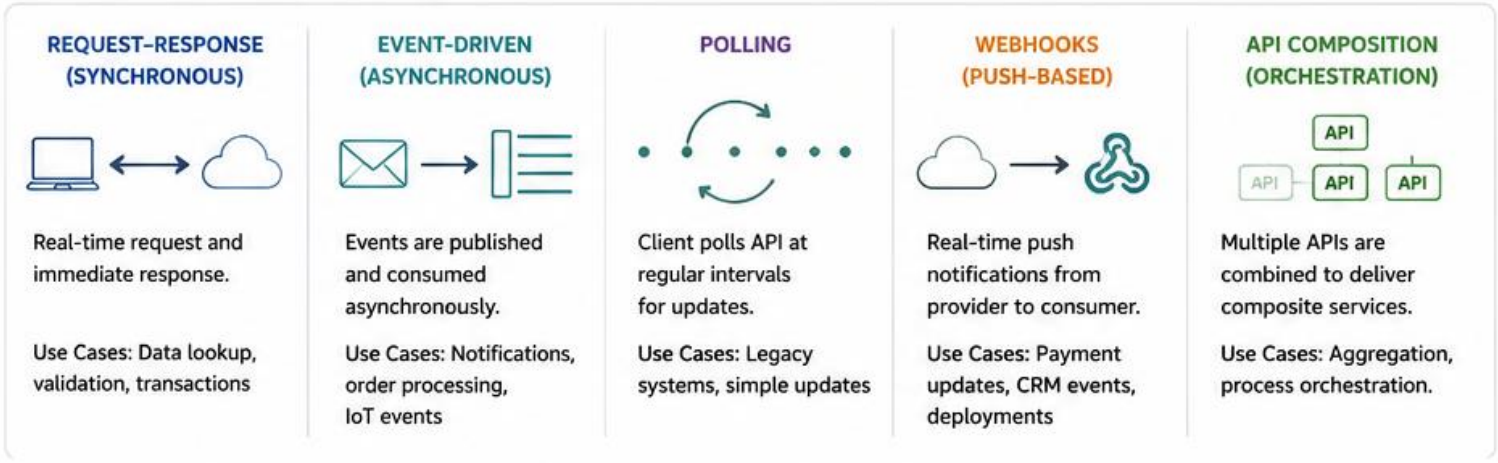
Common Use Cases

- On-premises to Cloud Integration
- Application Integration
- Partner / B2B Integrations
- Mobile & Web App Integration
- Legacy System Modernization
- Data Sharing & Analytics

HOW API-BASED INTEGRATION WORKS



COMMON API INTEGRATION PATTERNS

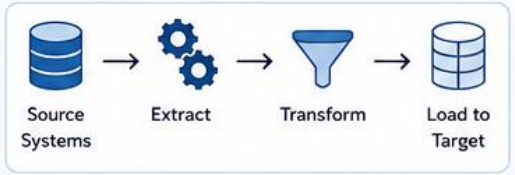


ETL, ELT, AND DATA REPLICATION

Three powerful approaches to move and integrate data—each with a unique purpose and use case.

- ETL (Extract, Transform, Load)
- Source Systems → Extract → Transform → Load to Target
- ELT (Extract, Load, Transform)
- Source Systems → Extract → Load to Target → Transform (in Target)
- Data Replication (Continuous Data Sync)
- Source Systems → Continuous Capture → Replicate / Sync → Target Systems

ETL
(Extract, Transform, Load)



OVERVIEW
Data is extracted from source systems, transformed in an intermediate staging area, and then loaded into the target system.

KEY CHARACTERISTICS
- Transformations happen before loading to target
- Requires staging area / ETL server
- Ideal for data cleansing, standardization, and enrichment
- Works well with smaller to medium data volumes
- Strong data quality control

BEST USE CASES
- Reporting data marts
- Legacy system migrations
- Data quality and standardization projects
- When target system has limited compute power

TOOLS (EXAMPLES)
AWS Glue, Informatica, Talend, SSIS, Pentaho, Apache NiFi

ELT
(Extract, Load, Transform)

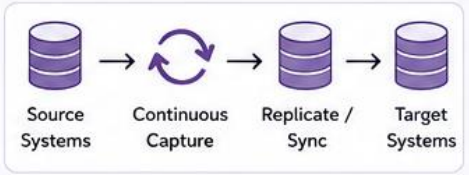
OVERVIEW
Data is extracted from source systems, loaded into the target (data lake / data warehouse), and transformations are performed in the target.

KEY CHARACTERISTICS
- Load raw data first, transform using target compute
- Leverages powerful compute of data lake/warehouse
- Faster loading, scalable for large volumes
- Keeps raw data for flexibility and reusability
- Ideal for modern cloud data platforms

BEST USE CASES
- Cloud data warehouses and data lakes
- Big data and analytics workloads
- Exploratory analytics and data science
- When transformations change frequently

TOOLS (EXAMPLES)
AWS Glue (ELT), Snowflake, Databricks, Google BigQuery, Azure Synapse, dbt

Data Replication
(Continuous Data Sync)



OVERVIEW
Data changes in the source are captured and continuously replicated to target system(s) with minimal latency.

KEY CHARACTERISTICS
- Continuous or near real-time synchronization
- Captures inserts, updates, deletes (CDC)
- Minimal transformation (if any)
- Low latency, high availability
- Supports operational and analytical use cases

BEST USE CASES
- Real-time reporting and analytics
- Operational system sync (Active-Active / DR)
- Microservices and distributed architectures
- Data availability and disaster recovery

TOOLS (EXAMPLES)
AWS DMS, Debezium, Fivetran, Qlik Replicate, Oracle GoldenGate

CHANGE DATA CAPTURE (CDC)

Change Data Capture (CDC) is a method of identifying and capturing data changes (inserts, updates, deletes) as they occur in the source system and delivering them to target systems with minimal latency.

Common Use Cases

- Real-time Analytics
- Data Synchronization
- Data Migration & Replication
- Event-Driven Architectures
- Operational Monitoring

WHAT IS CDC?

Change Data Capture (CDC) is a method of identifying and capturing data changes (inserts, updates, deletes) as they occur in the source system and delivering them to target systems with minimal latency.

KEY BENEFITS

- Near real-time data availability
- Minimal impact on source systems
- Supports real-time analytics and reporting
- Enables data synchronization & replication
- Improves data consistency across systems
- Reduces batch processing windows

HOW CDC WORKS

1. Detect
Identify data changes in the source database

2. Capture
Capture the change details (what, when, where)

3. Deliver
Deliver changes to target via stream or queue

4. Consume
Apply changes to target system(s)

CDC IMPLEMENTATION MODES

Log-Based CDC (Most Common)
- Reads database transaction logs (e.g., Oplog in MongoDB, WAL in PostgreSQL)
- Captures all data changes with high fidelity
- Low latency, high performance

Trigger-Based CDC
- Uses database triggers to capture changes
- Stores changes in audit/change tables
- Easy to implement, higher overhead on source

Query-Based CDC
- Periodically queries source tables for changes
- Based on timestamps or version columns
- Simple but higher latency and may miss edge cases

EXAMPLE CDC EVENT (JSON)

```
{
  "operation": "update",
  "source": "orders",
  "ts_ms": 1716304805123,
  "before": {
     "orderId": 1001,
     "status": "PENDING",
     "amount": 250.00
   },
   "after": {
     "orderId": 1001,
     "status": "CONFIRMED",
     "amount": 250.00
  }
}
```

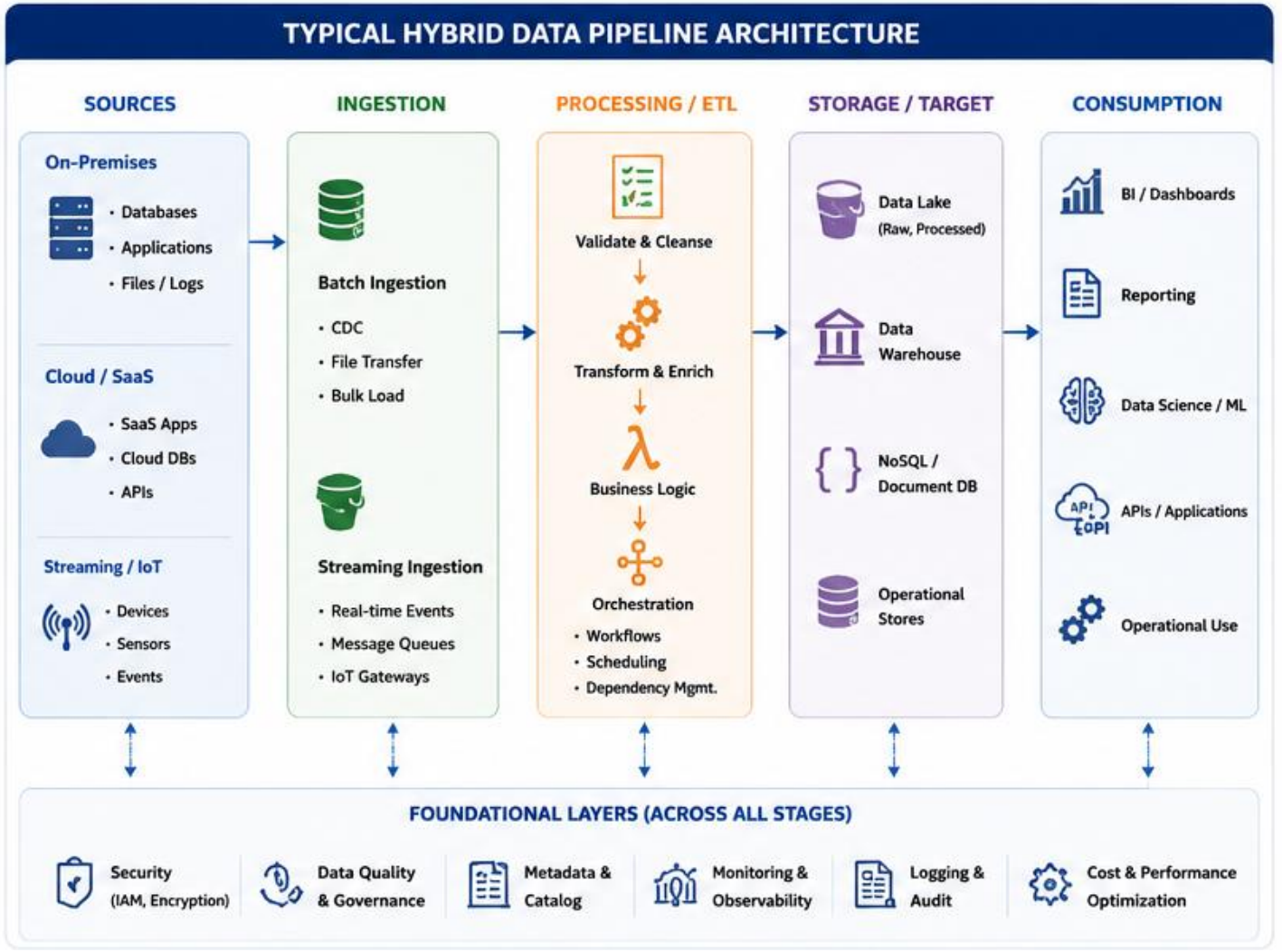
COMMON CDC TOOLS & PLATFORMS

Debezium	Open-source CDC platform for multiple databases; integrates with Kafka
AWS DMS	Managed CDC for migrating and replicating data to AWS and beyond
MongoDB Change Streams	Built-in CDC capability for real-time change feed in MongoDB
Apache Kafka	Stream platform commonly used to transport CDC events
Confluent	Enterprise Kafka platform with connectors for CDC pipelines

DESIGNING HYBRID DATA PIPELINES

Build scalable, secure, and reliable pipelines that move and transform data across on-premises and cloud environments to drive business value.

A hybrid data pipeline connects data sources in on-premises, cloud, and SaaS systems, processes and transforms the data, and delivers it to enterprise platforms for analytics, ML, reporting, and operational use.



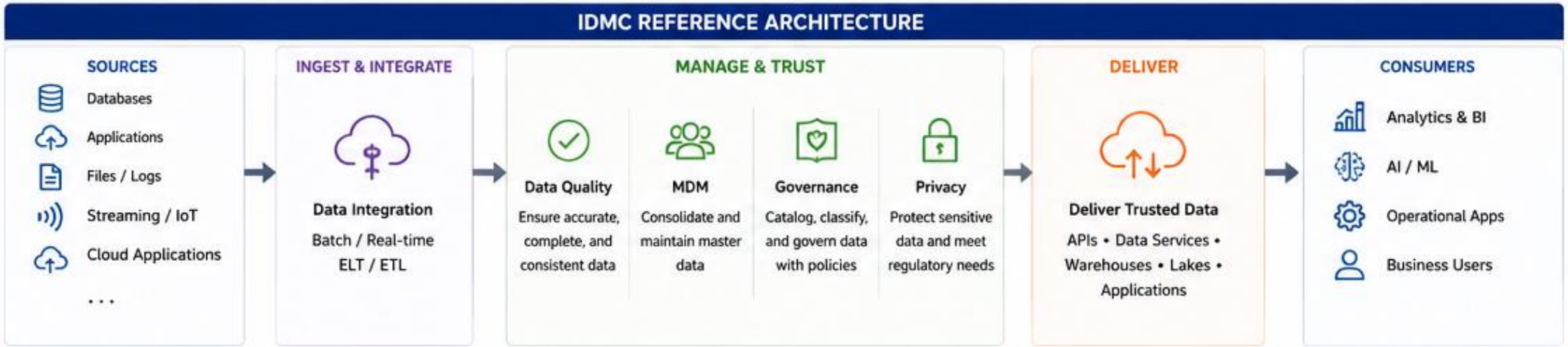
INFORMATICA IDMC OVERVIEW

Informatica Intelligent Data Management Cloud (IDMC) is a comprehensive, AI-powered cloud platform for data integration, quality, governance, and master data management.

IDMC is a multi-tenant, cloud-native platform that enables organizations to manage data across its lifecycle and deliver trusted data for analytics, AI, and business operations.

Core Pillars

- Cloud-Native
- AI-Powered
- Trusted Data
- End-to-End Platform



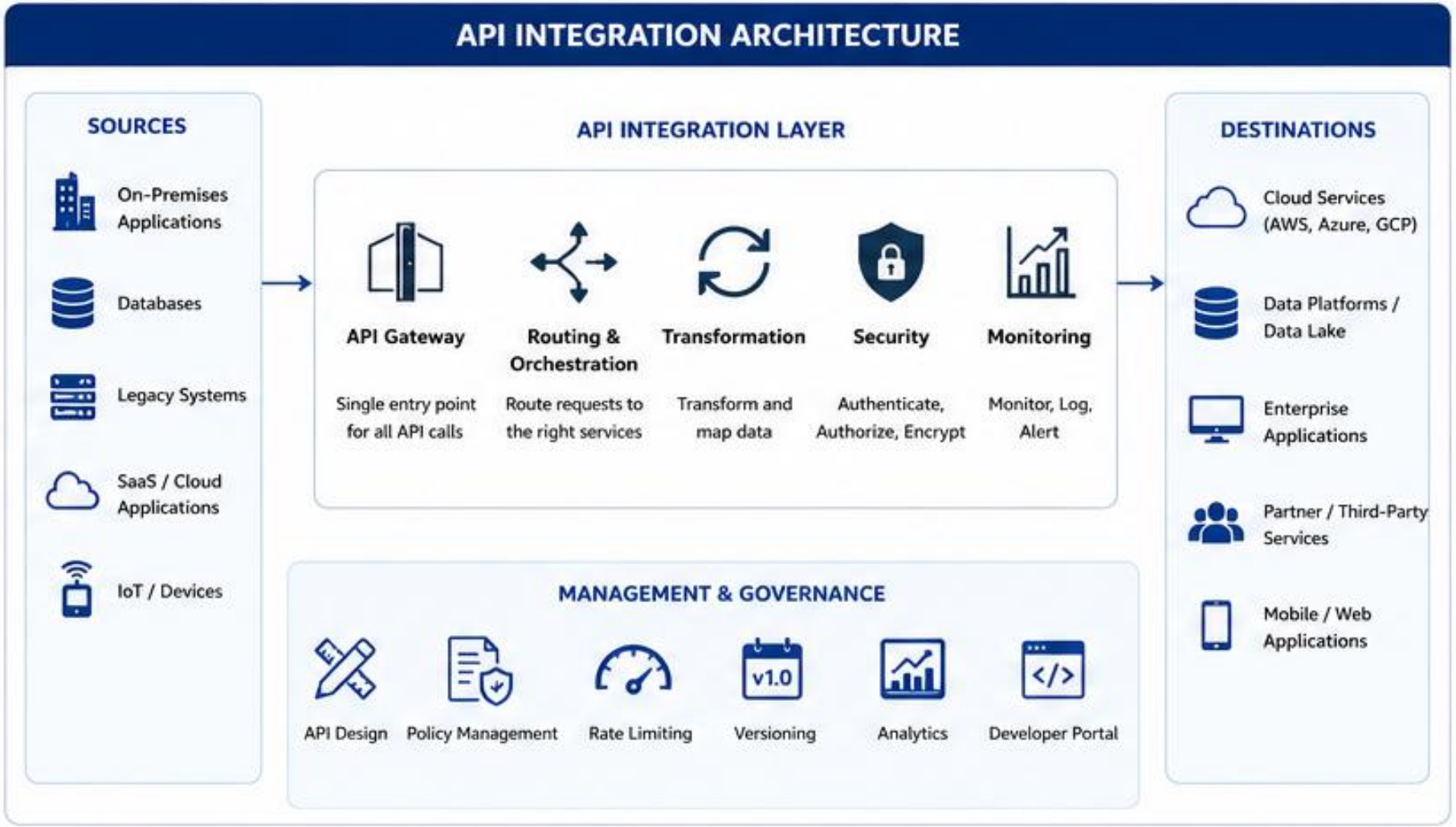
API INTEGRATION SERVICES

API Integration Services enable applications, systems, and data sources to securely exchange data and functionality using standard APIs (REST, SOAP, GraphQL, etc.).

They act as a bridge between diverse environments—on-premises, cloud, and SaaS—to support seamless interoperability.

Common Use Cases

- System-to-System Integration
- E-Commerce & Payment Integrations
- Data Synchronization Across Platforms
- IoT Data Ingestion via APIs
- Partner & Third-Party API Integrations
- API-Enabled Analytics and Reporting



METADATA MANAGEMENT

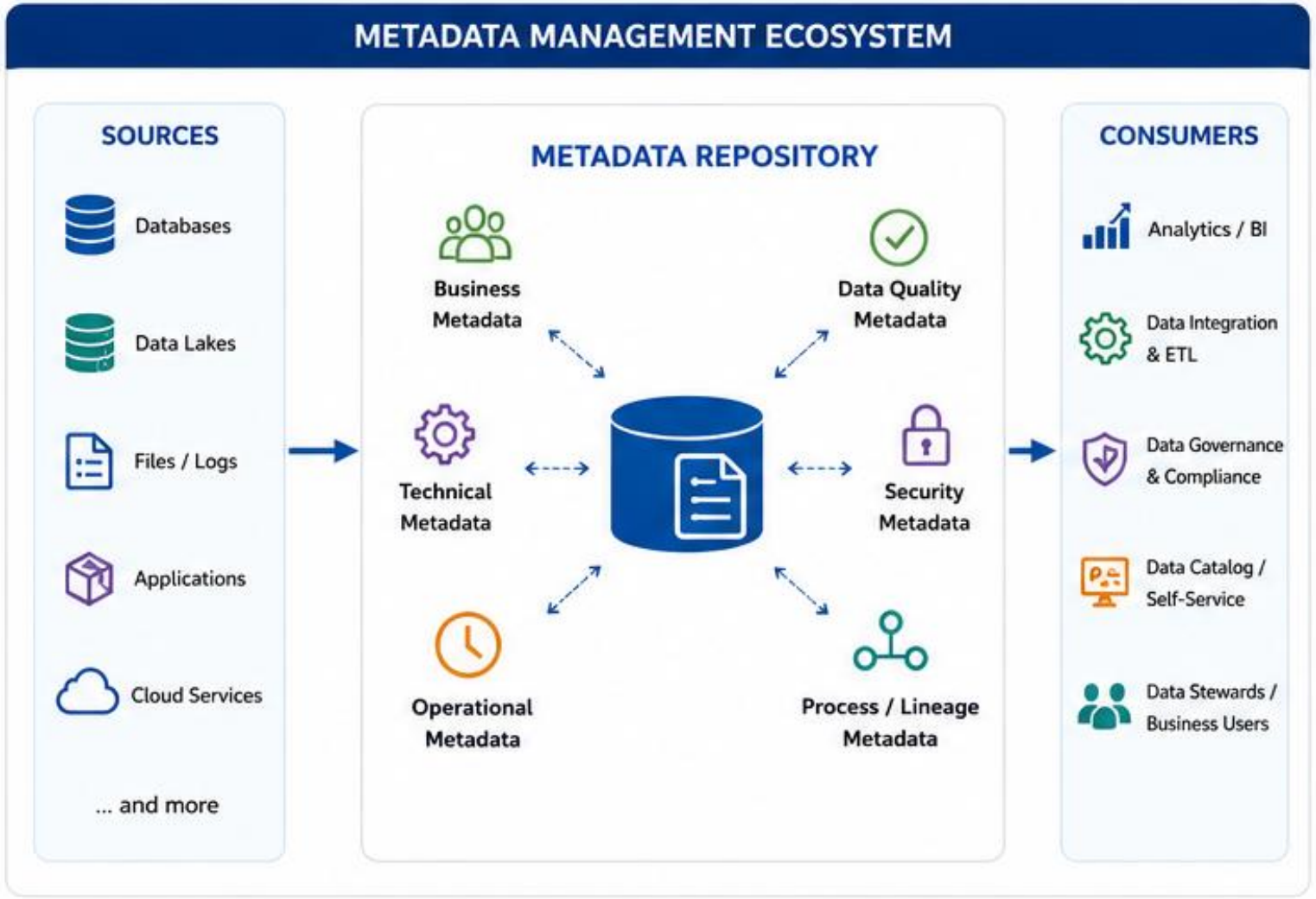
The foundation of trusted data, effective integration, and data-driven decisions.

Metadata is data about data. It describes the structure, meaning, origin, usage, and quality of data across systems and processes.

Effective metadata management provides visibility, context, and governance for the entire data lifecycle.

Metadata Value Pillars

- Discover – Understand your data
- Trust – Improve data quality & governance
- Integrate – Enable seamless data integration
- Empower – Drive analytics and innovation



MONITORING AND OBSERVABILITY

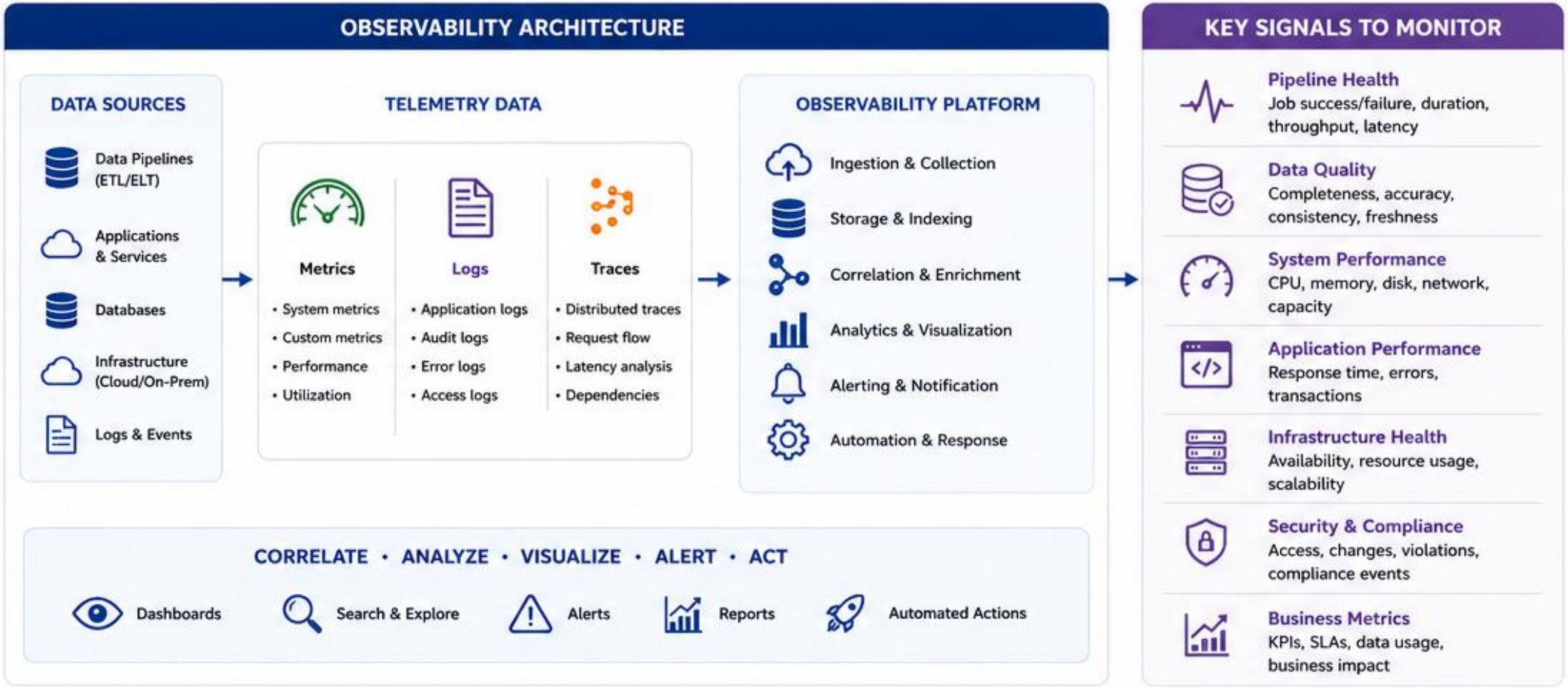
See everything. Understand deeply. Act quickly.

Monitoring tells you what is happening in your systems using predefined metrics and alerts.

Observability helps you understand why it is happening by giving you deep visibility into metrics, logs, traces, and business context.

Core Objectives

- Observe – Gain full visibility across systems
- Understand – Correlate metrics, logs and traces
- Detect – Proactively detect issues and anomalies
- Act – Respond faster and improve continuously



CUSTOMER 360 ARCHITECTURE

Customer 360 is a unified view of customer data from all sources, providing a complete, real-time understanding of each customer's interactions, preferences, and behavior across the entire lifecycle.

Strategic Outcomes

- Unified View – One customer across all touchpoints
- Personalized – Relevant experiences in real time
- Insights – Better decisions with deeper insights
- Business Value – Higher loyalty, retention, and satisfaction

KEY ARCHITECTURAL PRINCIPLES

Unified Identity
Resolve and link customer identities across all sources

Data Quality
Ensure accurate, complete, and consistent data

Real-time & Batch
Support real-time and batch data processing

Scalability
Design for growth and high performance

Security & Privacy
Protect customer data and ensure compliance

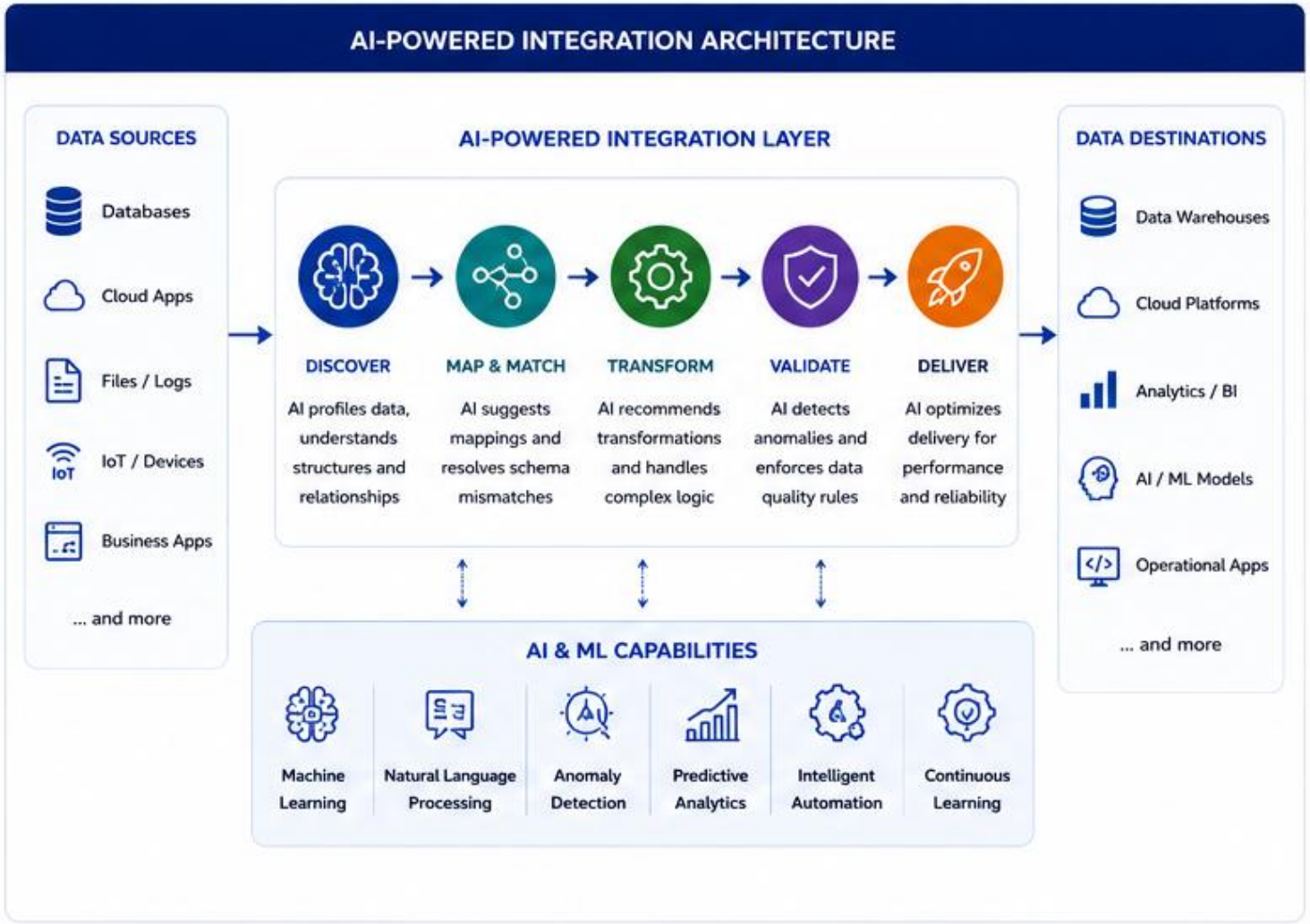
Modularity
Use reusable, decoupled components and APIs

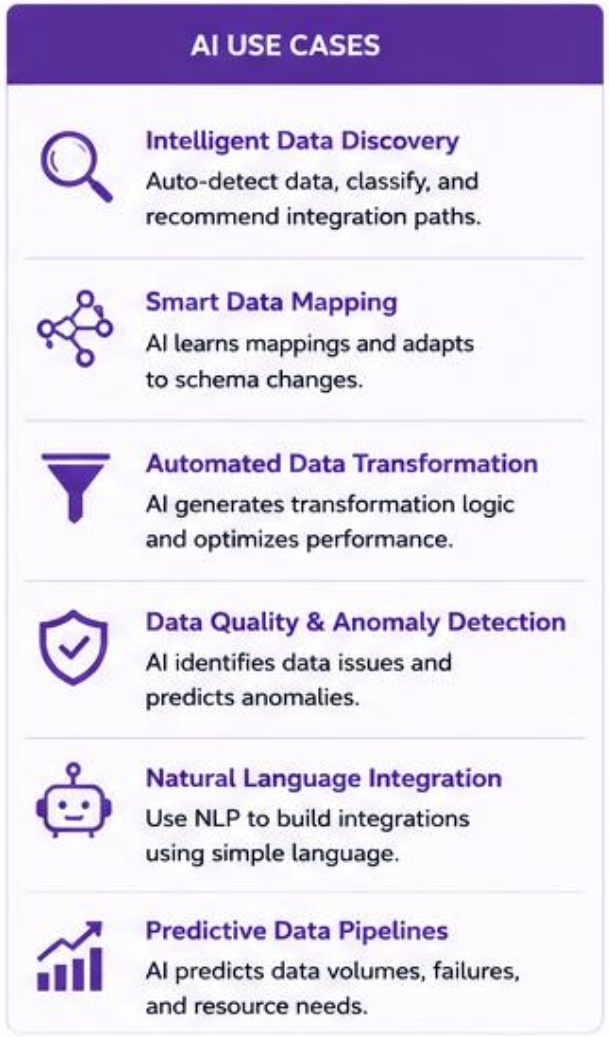
AI-POWERED DATA INTEGRATION

AI-Powered Data Integration leverages machine learning, natural language processing, and intelligent automation to connect, transform, and deliver data faster, with higher accuracy and greater adaptability. It moves integration from rule-based to intelligent and self-improving.

Strategic Benefits

- Intelligent – AI automates and optimizes integration processes
- Accurate – AI improves data quality and reduces errors
- Agile – AI accelerates time to insight and adaptation
- Impactful – AI drives better decisions and business outcomes





FUTURE OF ENTERPRISE DATA INTEGRATION

Intelligent. Real-Time. Autonomous. Everywhere.

Key Trends Shaping The Future

- AI-Native Integration: AI/ML for automation, mapping, data quality, and anomaly detection.
- Real-Time Everywhere: Streaming-first architectures for instant insights and actions.
- Cloud & Hybrid by Design: Seamless integration across multi-cloud, on-prem, and edge.
- Data Mesh & Federated Models: Domain-oriented ownership with governed data products.
- Trust & Governance at the Core: Privacy, security, and compliance built in by design.

THE EVOLVING INTEGRATION LANDSCAPE

TRADITIONAL
2000s

- Point-to-point integrations
- Batch processing
- On-premise centric
- Siloed data and teams

DIGITAL TRANSFORMATION
2010s

- SOA, ESB, iPaaS emergence
- Cloud adoption begins
- API-led connectivity
- More data, more sources

INTELLIGENT INTEGRATION
2020s

- API-first & event-driven
- Real-time & streaming
- AI/ML assisted integration
- Governed & scalable platforms

AUTONOMOUS INTEGRATION
FUTURE

- Self-learning & self-healing
- Autonomous data pipelines
- Context-aware data delivery
- Business outcome focused

FUTURE INTEGRATION CAPABILITIES

Intelligent Automation
AI-driven mapping, orchestration & testing

Event-Driven
Real-time streaming & micro-batching

Composable & API-Centric
Reusable, modular integration

Data Observability
End-to-end visibility, quality & lineage

Edge & IoT
Integration at the edge for real-time actions

Low Code / No Code
Faster development & citizen integration

THE BUSINESS IMPACT

Faster time-to-insight
Real-time data availability accelerates decision making

Greater Agility
Adapt quickly to change with scalable integration

Superior Customer Experience
Deliver personalized, seamless experiences across channels

Operational Efficiency
Automate processes and reduce integration complexity & cost

Data-Driven Innovation
Trusted data foundation fuels AI, analytics and new services

THANK YOU!